从传统机器学习到预训练 Transformer：音乐流派分类的系统性基准与优化

《走进音乐人工智能》课程项目组

代码与结果：https://github.com/weiza0v0/music-ai-lab

2026 年 10 月

摘要

音乐流派分类是音乐信息检索的经典任务，也是检验不同建模范式的天然试验台。本文在同一套数据划分、同一批音频片段与同一套评估协议下，系统比较了六类模型：基于人工特征的 SVM 与 MLP、端到端 CNN、CNN+BiGRU（CRNN）、轻量 Transformer，以及冻结 AudioSet 预训练主干的 Audio Spectrogram Transformer（AST）。实验覆盖两个数据集：清洗后的 GTZAN（985 首、10 流派，作为最小可行 MVP）与 FMA-small 主实验（8 000 首、8 流派）。所有深度模型使用 3 个随机种子重复训练，测试集只在模型冻结后评估一次，并启用了 cuDNN 确定性以保证可复现。在 GTZAN 上，从头训练的 CNN 的 Macro-F1 为 0.753；冻结预训练主干的 AST 达到 0.814；在 FMA-small 上最佳模型 AST（预训练）的 Macro-F1 为 0.590。本文进一步给出错误分析、七种扰动下的鲁棒性实验、片段级时间轴分析，以及面向 CNN/CRNN/Transformer 的 SpecAugment 与标签平滑优化消融。结果显示：预训练表示带来最大的单点收益，但在噪声与短片段下所有模型的可靠性都显著下降；深度模型赢在干净输入，传统模型在输入变脏时反而更稳。本文因此分为判别式与生成式两个任务：前者即上述流派分类，后者把预测目标从流派标签换成离散 **Token** 序列——用 EnCodec 把音频量化成 50 Hz、4 码本的 Token，比较 LSTM 与小 Transformer 对下 1 秒 Token 的预测能力，并评估 MusicGen-small 的前缀续写与 LoRA 适配。该部分记录了一次被修正的负结果：初版 LoRA 适配因标签维度错误而完全无效（目标值停在均匀分布之上），修正后音频距离从 19.23 降到 17.91，且 8 首中 7 首改善——说明” 训不动” 未必是容量或数据不足，也可能只是目标对齐写错了。代码、权重生成脚本与全部结果文件均已开源。

关键词：音乐流派分类；GTZAN；FMA-small；卷积神经网络；CRNN；Transformer；迁移学习；可复现实验

目录

1 引言 **4**
1.1 研究背景 . . . 4
1.2 问题与挑战 . . . 4
1.3 研究问题 . . . 4
1.4 主要贡献 . . . 5
1.5 报告组织 . . . 5

2 相关工作 **5**
2.1 音乐信息检索与流派分类 . . . 5
2.2 人工特征与浅层模型 . . . 5
2.3 端到端深度模型 . . . 5
2.4 Transformer 与预训练 . . . 6
2.5 数据集与评测偏差 . . . 6
2.6 鲁棒性与可复现性 . . . 6
2.7 离散音频 Token 与音乐生成 . . . 6

3 数据与实验协议 **6**
3.1 GTZAN（MVP 数据集） . . . 6
3.2 FMA-small（主实验数据集） . . . 7
3.3 统一预处理 . . . 8
3.4 划分协议与泄漏诊断 . . . 8
3.5 防泄漏与可复现约束 . . . 9
3.6 两个任务的协议共享与差异 . . . 9

I 判别式任务：音乐流派分类 **10**

4 方法与实验设置 **11**
4.1 传统基线：SVM 与 MLP . . . 11
4.2 CNN . . . 11
4.3 CRNN . . . 11
4.4 轻量 Transformer . . . 11
4.5 AST 迁移学习 . . . 11
4.6 训练细节与优化 . . . 11
4.7 训练算法 . . . 12
4.8 优化策略 . . . 12
4.9 复杂度与实现细节 . . . 12
4.10 软硬件环境 . . . 12
4.11 评估指标 . . . 13
4.12 重复实验与统计 . . . 13
4.13 实验矩阵 . . . 13
4.14 可复现性措施 . . . 13

5 结果与分析 **14**
5.1 GTZAN：最小可行 MVP . . . 14
5.2 FMA-small：主实验 . . . 16
5.3 优化消融 . . . 18

5.4 训练预算消融 . . . 19
5.5 计算成本与部署取舍 . . . 19
5.6 错误分析 . . . 20
5.7 鲁棒性 . . . 20
5.8 片段级分析 . . . 21

II 生成式任务：离散 Token 预测与音频续写 22

6 生成式任务：离散 Token 预测与音频续写 23
6.1 任务定义与动机 . . . 23
6.2 离散音频 Token：EnCodec 32 kHz . . . 23
6.3 数据窗口与泄漏控制 . . . 23
6.4 预测模型 . . . 24
6.5 训练与评价口径 . . . 24
6.6 Token 预测结果 . . . 24
6.7 把 Token 解码回音频 . . . 26
6.8 MusicGen-small 的前缀续写 . . . 27
6.9 参数高效适配：LoRA . . . 27
6.10 小结与局限 . . . 28

III 综合讨论与结论 29

7 讨论 30
7.1 主要发现 . . . 30
7.2 判别式与生成式：同一种纪律，两种可度量性 . . . 30
7.3 负结果是怎么被辨别出来的 . . . 30
7.4 预训练的收益与代价 . . . 31
7.5 为什么从头训练的 Transformer 会输 . . . 31
7.6 数据规模与任务难度 . . . 31
7.7 鲁棒性的进一步解读 . . . 31
7.8 威胁与局限 . . . 32
7.9 伦理与数据授权 . . . 32
7.10 可复现性总结 . . . 32
7.11 对课程项目的启示 . . . 32

8 结论与展望 33

A 附录：数据、超参与复现细节 34
A.1 数据集统计 . . . 34
A.2 完整超参数 . . . 35
A.3 鲁棒性完整变化表 . . . 35
A.4 AST 的 C 网格 . . . 35
A.5 仓库结构与复现命令 . . . 35
A.6 结果文件说明 . . . 37

1 引言

1.1 研究背景

音乐流派（genre）是人类对音乐风格的一种高层概括，广泛用于推荐、检索、曲库管理与版权结算。自动流派分类要求模型从波形中提取音色、节奏、和声与配器等线索，是连接信号处理与音乐理解的典型任务 [1, 13]。在流媒体平台上，每天有大量新曲目缺少人工标注，自动流派标签能显著降低冷启动成本；在音乐教育中，流派分类也可以作为理解音色与节奏的入口。

过去十余年，该任务的建模路线经历了三次明显的迁移：从人工特征加浅层分类器，到端到端卷积网络 [4, 14]，再到自注意力 [6] 与大规模预训练模型 [5, 15]。每一次迁移都伴随新的问题：深度模型是否真的优于精心设计的人工特征？预训练带来的收益有多少来自数据、多少来自架构？在资源受限的课堂上，我们能否得到可信、可复现的结论？

与分类这条线并行的，是近三年兴起的生成式路线。其关键一步是把连续波形离散化：神经音频编解码器（SoundStream、EnCodec[21]）用残差向量量化把音频压成每秒几十帧、每帧若干个码本的离散 Token。一旦音频变成 Token 序列，”下一个 Token 是什么”就成为一个可以用交叉熵直接度量的语言建模问题——AudioLM[22] 据此做音频续写，MusicGen[23] 进一步把文本条件接进来，成为可控的音乐生成模型。这条路线带来一个诱人的猜想：如果同一个系统既能把音频压成 Token，又能预测 Token，那么”理解音乐”与”生成音乐”或许是同一件事的两面。

本文因此在流派分类之外，用同一批数据与同样严格的划分纪律再做一个生成式任务：在离散 Token 层面预测真实未来、把预测解码回音频、并评估预训练生成模型的前缀续写。两个任务并列，而不是把生成当作分类的附属演示。

1.2 问题与挑战

文献中的准确率数字往往不可直接比较，原因至少包括：

1. 数据划分不一致：同一数据集的不同版本、不同随机划分，甚至测试集被反复用于调参；
2. 片段与预处理不一致：3秒 / 5 秒 / 10 秒片段、采样率、归一化方式都会改变输入分布；
3. 标签本身模糊：「Rock」与「Metal」、「Pop」与「Dance」的边界在人类标注者之间也不一致；
4. 数据泄漏隐蔽：同一曲目的不同片段、同一艺术家的不同作品跨集合，都会让模型「记住来源」而不是学习风格；
5. 计算预算差异：预训练模型的权重体积与推理延迟比自建模型高两到三个数量级。
6. 生成任务没有唯一的正确答案：分类有明确标签可以对齐，而”这段前缀之后应当是什么”本身是开放集合。用与真实后续的距离（如 Log-Mel RMSE 或 FAD[25]）度量，只能说明”接近真实”而非”好听”；主观评分又难以规模化。因此生成实验必须明确区分可度量的部分（Token 级 NLL、Top-1）与只能定性判断的部分（音乐性）。

因此，一个负责任的基准必须同时报告协议、成本与失败模式，而不只是一个准确率。

1.3 研究问题

本文围绕三个问题展开：

1. 在完全相同的协议下，不同代际的模型（传统 ML、CNN、CRNN、Transformer、预训练 AST）差距究竟有多大？
2. 数据规模从千首级（GTZAN）扩大到万首级（FMA-small）时，结论是否稳定？
3. 常见优化手段（SpecAugment、标签平滑、dropout）能带来多少真实收益，代价是什么？
4. 把目标从流派标签换成离散 Token 序列后，同一套小容量模型与评估纪律能得到什么？8 GB 显存的课堂上，预训练生成模型的前缀续写与轻量适配各自能做到哪一步，失败的边界又在哪里？

1.4 主要贡献

本文的主要贡献如下：

- 统一基准：在同一套按曲目 / 按艺术家划分、同一批 5 秒中心片段、同一套预处理与评估协议下，比较六个代表性模型；
- 泄漏诊断：量化了 FMA-small 上「按曲目划分」带来的艺术家级泄漏，并用对照实验证明其会把 CNN 的 Macro-F1 虚高约 9.9 个百分点；
- 优化消融：系统尝试 SpecAugment、标签平滑与 dropout，报告了一个诚实的负结果——在 FMA 上最大提升仅 0.7 个百分点；
- 行为分析：给出错误分析、七种扰动的鲁棒性、片段级时间轴与置信度校准，说明准确率之外的模型差异；
- 可复现交付：公开代码、配置、结果文件与 LaTeX 报告，所有数字可由脚本重新生成，且经过独立只读审计。
- 生成式任务基准：用 EnCodec 把音频量化为 4×300 Token，在同一份按艺术家分离的划分上比较 Persistence、频率先验、Bigram 与两种神经模型（LSTM、小 Transformer），并分别报告 teacher-forced 与 free-running 两种口径，避免用前者冒充后者。
- 一次被修正的负结果：预训练生成模型的 LoRA 适配初版完全无效。我们用未训练模型的目标值对照（错误布局 11.08 / 正确布局 5.46 / 均匀分布 7.62）定位到标签维度错误，修正后音频距离由 19.23 降到 17.91 且 8 首中 7 首改善；而随后对训练步数、目标窗口、适配秩三个旋钮的配对检验均不显著。
- 负结果的辨别方法：本文给出三件在报告中容易被忽略的检验——逐曲配对检验（排除” 单首拉低均值”）、去掉最优样本后的复核、以及按几何衰减外推判断” 最优轮次撞上限” 是否真的意味着欠训练。三者都曾改变本文的结论。

1.5 报告组织

第 2 节回顾相关工作；第 3 节介绍数据与预处理协议。此后正文分为三个部分。第一部分处理判别式任务：第 4 节给出模型、训练方法与实验设置，第 5 节报告 GTZAN 与 FMA-small 的结果、消融，以及错误、鲁棒性与片段级分析。第二部分处理生成式任务：第 6 节把目标从流派标签换成离散 Token 序列，依次给出 Token 化协议、预测模型、预测结果、解码验证，以及预训练生成模型的前缀续写与轻量适配。第三部分是综合讨论：第 7 节汇总主要发现、解释性分析与局限，第 8 节总结。附录给出逐 seed 结果、完整超参与复现命令。

2 相关工作

2.1 音乐信息检索与流派分类

音乐信息检索（Music Information Retrieval, MIR）关注从音频、乐谱、歌词或元数据中提取结构化信息 [16, 13]。流派分类是其中最早被系统研究的任务之一：Tzanetakis 与 Cook 在 2002 年提出用谱特征、节奏特征与梅尔倒谱系数训练分类器，并发布了 GTZAN 数据集 [1]。此后二十年，该任务逐渐成为检验新表示与新架构的标准试验台。

2.2 人工特征与浅层模型

传统方法通常把音频切帧，逐帧计算 MFCC、谱质心、谱带宽、rolloff、过零率、chroma 与节奏直方图，再在时间维做统计池化，最后送入 SVM、随机森林或 k 近邻 [1, 17]。这类方法有两个优点：一是计算便宜，二是特征具有明确的声学含义。其局限在于统计池化会丢失时间结构，且特征设计依赖领域知识。

2.3 端到端深度模型

随着数据规模与算力增长，端到端学习逐渐取代人工特征。Dieleman 与 Schrauwen 直接把波形送入 CNN[14]；Choi 等提出 CRNN，用卷积层提取局部时频模式、用循环层建模时间演化 [4]；Humphrey 等系统比较了特征

学习与手工特征 [19]。在音频事件识别领域，Hershey 等比较了多种 CNN 架构 [20]，PANNs 则用大规模预训练提升通用性 [15]。

2.4 Transformer 与预训练

Transformer 的自注意力机制 [6] 能建模长程依赖，Vision Transformer 把图像切 patch 后直接用 Transformer 编码。AST 把这一思路迁移到音频：将 Log-Mel 频谱切成 16×16 的 patch，在 AudioSet[7] 上预训练，再迁移到下游分类任务 [5]。在小数据场景，冻结主干、只训练线性分类头是常见策略，但其收益同时来自预训练数据与架构，两者需要分开讨论。

2.5 数据集与评测偏差

GTZAN[1] 是最常用的基准，但 Sturm 指出其存在重复音频、标签错误、失真与艺术家泄漏等问题，容易高估模型能力 [2]。FMA[3] 提供了规模更大、授权更清晰的替代数据；MSD[18] 则侧重百万级曲目与元数据。本文选择 GTZAN 作为快速验证集、FMA-small 作为主实验集，并对两者都做泄漏检查。

2.6 鲁棒性与可复现性

真实部署中的音频会经历音量变化、噪声、压缩与片段截断。已有工作表明，音频分类模型对这些扰动并不稳健，且不同特征表示（绝对 dB 与归一化 dB）的敏感性差异很大。本文用七种扰动系统评测五个模型，并把可复现性作为一等公民：固定种子、确定性 kernel、自动化协议测试与独立审计。

2.7 离散音频 Token 与音乐生成

要把音乐生成交给语言模型，第一步是把连续波形变成离散序列。SoundStream 与 EnCodec[21] 用残差向量量化（RVQ）把音频压成低帧率的码本序列：EnCodec 在 32 kHz 上以 50 Hz 输出 4 个码本、每个码本 2048 个词表项，码率约 2.2 kbps，解码后的听感仍接近原始音频。这一步的意义不只是压缩——它让音频与文本一样，成为” 可以用交叉熵预测的离散序列”。

AudioLM[22] 直接利用这一点：把音频分成语义 Token 与声学 Token 两级，用 Transformer 做自回归续写，得到长时间尺度的连贯音频。MusicLM 把文本条件加入同一框架。MusicGen[23] 做了简化：单阶段 Transformer 直接建模 EnCodec 的多个码本，用 **delay pattern** 把并行的码本流错开一帧、拼成一条序列，从而用一个因果语言模型同时处理多个码本；其 small 版本约 300 M 参数，是本项目在 8 GB 显存上唯一可行的选择。本项目的生成式任务沿用这套 Token 格式，因此需要先实测本环境的 EnCodec 与 MusicGen 权重是否真的互操作（能解码回放才算兼容），而不是假设” 同系列即可互换”。

在适配层面，全量微调一个 300 M 的生成模型对 8 GB 显存并不划算。LoRA[24] 冻结原始权重、只训练低秩旁路（$W + \frac{\alpha}{r} BA$），把可训练参数压到原来的千分之一量级，是资源受限场景的常见选择。需要注意的是，LoRA 只降低优化成本，并不保证目标对齐正确——本文的生成式任务正是在这一点上踩过一个不报错的坑。

生成任务的评价本身仍有争议。Fréchet Audio Distance[25] 等参考无关指标用预训练嵌入的分布距离度量” 像不像真实音乐”，但它与人类主观判断的相关性并不稳定；与真实后续的频谱距离（本文采用的 Log-Mel RMSE）则只衡量” 接近这段真实音频”，不衡量音乐性。因此本文把 Token 级指标、音频级距离与定性试听分开报告，不把任何一个单独当成” 生成质量”。

3 数据与实验协议

3.1 GTZAN（MVP 数据集）

GTZAN 包含 10 个流派各 100 首、每首约 30 秒、22 050 Hz 单声道音频。审计发现 1 首无法解码（`jazz.00054`），28 首与其他曲目 MD5 完全相同。我们剔除损坏文件，并对重复组只保留第一首，最终得到 **985** 首。按曲目分层划分 70/15/15（689/149/147），随机种子 42。逐流派统计见表 1。

表 **1:** GTZAN 清洗后的逐流派统计。

流派	曲目数	平均时长 (s)	采样率
blues	100	30.01	22,050
classical	100	30.03	22,050
country	100	30.02	22,050
disco	99	30.02	22,050
hiphop	98	30.08	22,050
jazz	99	30.03	22,050
metal	93	30.00	22,050
pop	98	30.00	22,050
reggae	99	30.01	22,050
rock	99	30.02	22,050
合计	985	30.02	–

GTZAN 的优点是规模小、类别均衡、加载快；缺点是标签噪声与重复音频，且缺少可靠的艺术家元数据（原始文件仅以文件名区分曲目）。因此它适合做快速迭代与教学演示，不适合作为唯一证据。

3.2 FMA-small（主实验数据集）

FMA-small 包含 8 000 首、8 个均衡流派的 30 秒片段，原始采样率 44.1 kHz。初步按曲目分层 70/15/15 划分后，我们做了艺术家级泄漏检查，发现测试集中 971 / 1199 首（81.0%）与训练集共享艺术家——同一乐队的作品在音色、录音与编曲上高度相似，这会让模型「认出乐队」而不是「听懂流派」。因此主实验改用按艺术家分离的划分（每位艺术家整体进入且仅进入一个集合，测试集与训练集的艺术家重叠为 0），并按流派保持大致比例（由于艺术家作品数不均，逐类比例并不完全相等，见附录表 25）。曲目级划分保留为泄漏对照实验。数据审计还发现 6 首损坏音频（与官方 SHA1 清单一致，属上游数据问题）与 4 首中心 5 秒数字静音的曲目；前者剔除，后者保留并在附录中说明。艺术家分离划分的逐流派数量见表 2。

表 **2:** FMA-small 艺术家分离划分的逐流派数量。

流派	训练	验证	测试
Electronic	726	134	139
Experimental	733	119	147
Folk	614	249	137
Hip-Hop	694	168	135
Instrumental	792	82	126
International	650	194	156
Pop	689	120	191
Rock	698	133	168
合计	5596	1199	1199

需要强调，艺术家分离划分不是「免费」的：由于部分艺术家拥有大量作品（最多的一个艺术家有 208 首），划分后逐流派的验证/测试比例并不完全相等（例如 Folk 的验证集 249 首、Instrumental 的验证集 82 首）。这会轻微影响早停与阈值选择，但测试集仍有 126–191 首/类，宏平均指标对类别比例不敏感。

3.3 统一预处理

所有模型使用每首曲目中心 **5** 秒片段，重采样到 22 050 Hz 单声道。传统模型输入 209 维人工特征，深度模型输入 64×216 的 Log-Mel 频谱：

$$S_{\mathrm{dB}}(m,t) = 10\log_{10}\left(\sum_{f} |X(f,t)|^2 \, H_m(f)\right), \qquad \tilde{S} = S_{\mathrm{dB}} - \max_{m,t} S_{\mathrm{dB}}, \tag{1}$$

其中 H_m 为 64 个 Mel 滤波器，$|X(f,t)|^2$ 为短时傅里叶功率谱（$n_{\mathrm{fft}} = 1024$，hop$= 512$）。以片段最大值为参考的归一化使表示对整体增益不敏感，这一点在鲁棒性实验中会再次出现。

209 维人工特征的组成见表 3。所有统计量在时间维做均值与标准差池化；标准化器只在训练集拟合。

表 **3:** 209 维人工特征的组成。

特征组	说明	维度
MFCC（40）	均值 + 标准差	80
ΔMFCC（40）	一阶差分的均值 + 标准差	80
Chroma（12）	音高类能量均值 + 标准差	24
谱对比度（7）	均值 + 标准差	14
谱形状（3）	质心、带宽、rolloff 的均值 + 标准差	6
过零率	均值 + 标准差	2
RMS 能量	均值 + 标准差	2
Tempo	估计值（归一化）	1
合计		209

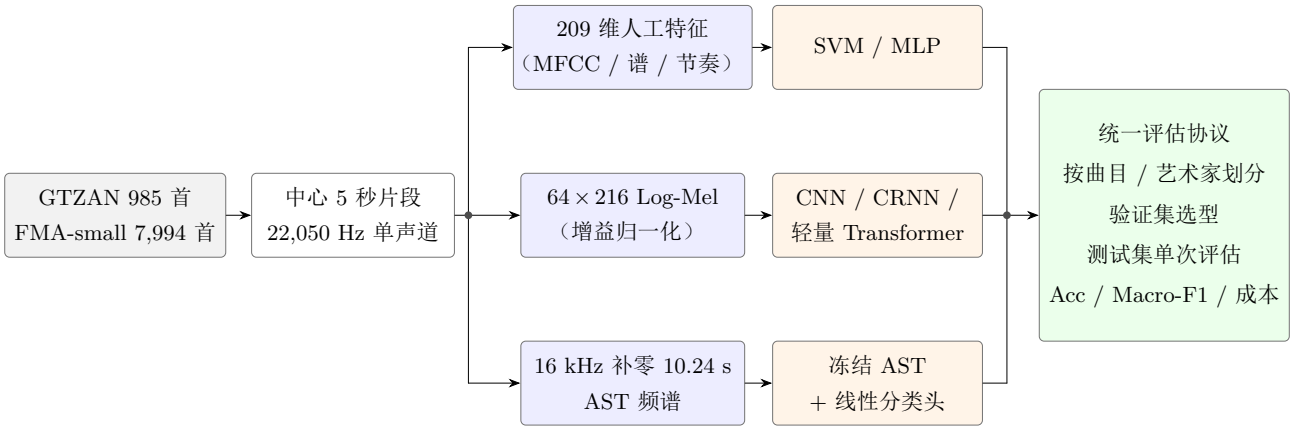


图 **1:** 统一的实验流程：同一份按曲目（FMA 按艺术家）划分的数据，经分发母线进入人工特征、Log-Mel 与 AST 三条支路，再由汇流母线进入统一评估协议；三条支路共享同一批 5 秒中心片段，所有预处理只在训练集上拟合。

3.4 划分协议与泄漏诊断

FMA-small 的元数据包含艺术家字段。我们先用常规的按曲目分层划分，然后检查测试集与训练集的艺术家重叠：971 / 1199 首（81.0%）测试曲目与训练集共享至少一位艺术家。同一乐队的作品在音色、录音设备与编曲习惯上高度相似，这会为模型提供一条「认乐队」而非「听懂流派」的捷径。

因此主实验采用按艺术家分离的划分：每位艺术家整体进入且仅进入一个集合。表 4 对比两种划分。可以看到，曲目级划分把 CNN 的 Macro-F1 从 0.470 抬到 0.568（+9.9 个百分点），而输入特征、模型与训练超参完全相同——这正是数据泄漏的典型表现。

表 **4:** 曲目级划分与艺术家分离划分的对比（FMA-small，CNN seed 42）。

划分方式	train / val / test	测试-训练共享艺术家	Acc	Macro-F1	结论
按曲目	5,595 / 1,200 / 1,199	971（81.0%）	0.566	0.568	高估
按艺术家	5,596 / 1,199 / 1,199	0（0.0%）	0.480	0.470	主表采用

3.5 防泄漏与可复现约束

- 先按曲目划分，再切片，同一曲目绝不跨集合；
- StandardScaler 与 Log-Mel 归一化统计量只在训练集上拟合；
- 模型选择只看验证集 Macro-F1，测试集在模型冻结后只评估一次；
- FMA 主实验按艺术家划分，避免同乐队跨集合；并用曲目级划分做泄漏对照；
- 固定种子（42/123/2024），开启 cuDNN 确定性，同种子重跑得到逐字节一致的权重；
- 协议由 `tests/test_protocol.py` 自动校验，并由独立审计复核。

3.6 两个任务的协议共享与差异

判别式与生成式两个任务共用同一份数据与同一条纪律，但在输入格式与评价口径上分道扬镳。表 5 汇总了两者的对应关系。

表 **5:** 两个任务在协议上的共享项与差异项。共享项保证” 同一套纪律”，差异项说明为什么两者的指标不能直接比较。

维度	判别式任务	生成式任务
数据来源	FMA-small（同 GTZAN 作 MVP）	同一份 FMA-small
划分	按艺术家分离，5,596/1,199/1,199	完全复用同一份划分
测试集使用	冻结后评一次	冻结后评一次
种子	42 / 123 / 2024	42 / 123 / 2024
输入	5 s 中心片段，22.05/16 kHz	6 s 窗口，32 kHz 单声道
表示	209 维人工特征 + 64 × 216 Log-Mel	4 × 300 离散 Token（50 Hz）
输出	10 或 8 类标签	4 个码本的 Token 序列
指标	Accuracy、Macro-F1、混淆矩阵	NLL、Top-1（两种口径）、Log-Mel RMSE

需要强调两点。其一，划分严格复用意味着两个任务的测试曲目完全相同——因此” 分类器认对了这首歌” 与” 模型能续写这首歌” 是对同一批曲目的两种提问，结论才有可比性。其二，两者指标不可直接比较：Macro-F1 是” 答对多少”，NLL 是” 分布有多接近”，Log-Mel RMSE 是” 生成的频谱与真实未来差多远”。本文在任何地方都不做跨任务的数值比较。

生成式任务的窗口构造有其特殊性：先在曲目/艺术家层面划集合，再在曲目内部切窗，本次采用每首 2 个 6 s 窗口（前 5 s 作前缀、后 1 s 作预测目标），共 15,988 个窗口。窗口继承曲目所属的集合，因此不存在同一首歌的窗口跨越训练/测试的泄漏。

I 判别式任务：音乐流派分类

4 方法与实验设置

4.1 传统基线：SVM 与 MLP

SVM 使用 RBF 核（$C = 10$，γ =scale），MLP 为两层全连接（256-128，ReLU，Adam，early stopping）。二者共享同一份 209 维特征，标准化器仅在训练集拟合。

4.2 CNN

输入 $1 \times 64 \times 216$ 的 Log-Mel。网络由 4 个卷积块组成，每块含两个 3×3 卷积、BatchNorm、ReLU 与 2×2 最大池化，通道数依次为 32/64/128/128；随后自适应平均池化到 2×4，dropout 后接线性分类头，参数量约 59 万。三个自建深度模型的层结构与参数量见表 6。

表 **6:** 自建模型的层结构与参数量（GTZAN 10 类；括号内为 FMA 8 类的差异）。

模型	关键层（输入 → 输出）	参数量
CNN	Conv3×3(32)×2 → Pool Conv3×3(64)×2 → Pool Conv3×3(128)×2 → Pool×2 AdaptiveAvgPool(2,4) → FC(1024,10)	592,554
CRNN	Conv(32/64/128) → BiGRU(96) 时间平均 → FC(192,10)	418,986
Transformer	Linear(64,128)+Pos → 2×Pre-LN Encoder 时间平均 → FC(128,10)	302,474

4.3 CRNN

CNN 前端（3 个卷积块，池化 $(2,2)/(2,2)/(2,1)$）将频率维压缩后，按时间维展开为序列，送入双向 GRU（隐藏维 96），对时间维取平均后分类。参数量约 42 万。

4.4 轻量 Transformer

将每一帧的 64 维 Mel 向量线性投影到 $d = 128$，加可学习位置编码，经过 2 层 Pre-LN Transformer（4 头，FFN 256），对时间维做平均池化后分类。模型刻意保持轻量，以匹配 8 GB 显存与课程算力预算。

4.5 AST 迁移学习

使用 `MIT/ast-finetuned-audioset-10-10-0.4593`（86M 参数，AudioSet 预训练）。音频重采样到 16 kHz、取中心 5 秒并补零到 10.24 秒，经 AST 特征提取器得到 128×1024 频谱；冻结主干，取 `pooler_output` 的 768 维表示，在训练集上训练 StandardScaler + 逻辑回归分类头（C 由验证集选择）。该组单独标记为「迁移学习组」。

4.6 训练细节与优化

深度模型统一使用 AdamW、余弦退火、batch size 32、最多 30 个 epoch，按验证集 Macro-F1 早停（patience 8）并保存最佳权重。针对小数据过拟合，我们引入两种正则化并在验证集上做消融：

- **SpecAugment**[8]：对归一化后的频谱随机遮挡 2 个频带（宽度 ≤ 8）与 2 个时间段（宽度 ≤ 16），遮挡值置为 0（即训练集均值）；
- 标签平滑：交叉熵目标改为 $(1-\epsilon)y + \epsilon/K$，$\epsilon = 0.05$；同时把 dropout 提高到 0.4。

所有消融只使用验证集选型，测试集不参与任何决策。

4.7 训练算法

所有深度模型共享同一套训练流程（算法 1）：AdamW 优化、余弦退火、按验证集 Macro-F1 早停、保存最佳权重、测试集只在训练结束后评估一次。该流程被封装在 `src/train_cnn.py` 中，模型之间的差异仅在网络结构与输入表示。

算法 1: 统一训练与评估流程

Input: 训练/验证/测试划分 D_{tr}, D_{va}, D_{te}；模型 f_θ；最大轮数 E；耐心 p

Output: 最佳权重 $\theta^\star$；测试指标

```
在 D_tr 上拟合标准化器 / 归一化统计量;
for e = 1 to E do
    打乱 D_tr，按 batch 采样;
    可选：对 Log-Mel 施加 SpecAugment;
    前向计算、交叉熵（可选标签平滑）、反向传播、AdamW 更新;
    在 D_va 上计算 Macro-F1 F_e;
    if F_e > F_best then
        θ* ← θ；F_best ← F_e；耐心计数清零;
    else
        耐心计数加 1;
    if 耐心计数 ≥ p then
        break;
加载 θ*，在 D_te 上评估一次并冻结结果;
```

4.8 优化策略

针对小数据过拟合，我们尝试两类正则化并在验证集上做消融：

- **SpecAugment**[8]：对归一化后的频谱随机遮挡 2 个频带（宽度 ≤ 8）与 2 个时间段（宽度 ≤ 16），遮挡值置为 0（训练集均值）；
- 标签平滑 [12]：把独热目标改为

$$q_k = (1-\epsilon)\, y_k + \frac{\epsilon}{K}, \tag{2}$$

其中 K 为类别数，$\epsilon = 0.05$；同时把 dropout 从 0.3 提高到 0.4。

所有消融只使用验证集选型，测试集不参与任何决策；结果见第 5 节。

4.9 复杂度与实现细节

表 6 给出了各模型的参数量：CNN 约 59 万、CRNN 约 42 万、Transformer 约 30 万，而预训练 AST 主干有 8,619 万参数。参数量并不直接决定推理成本：在 RTX 4060 上，CNN/CRNN 的单片段延迟约 0.7–0.8 ms，轻量 Transformer 约 0.26 ms，而 AST 为 231 ms（含 16 kHz 重采样与频谱特征化），相差约三个数量级。

实现上，所有音频特征与 Log-Mel 在每个数据集上只计算一次并缓存为 `.npz`，训练时直接读取，避免重复解码 MP3；DataLoader 使用 `num_workers=0` 并配合页锁定内存，保证可复现优先于吞吐；深度模型开启 cuDNN 确定性（见第 4 节）。在 8 GB 显存下，batch size 32 足以训练全部自建模型，峰值显存远低于上限。

4.10 软硬件环境

实验在 WSL2 Ubuntu 22.04 上完成，具体版本见表 7。GPU 为笔记本级 RTX 4060（8 GB），显存足以容纳全部自建模型与 AST 的批量推理。

表 **7:** 实验环境。

项目	版本 / 配置
操作系统	Ubuntu 22.04（WSL2，内核 6.18）
Python	3.12.13
PyTorch	2.12.1+cu130
CUDA / 驱动	CUDA 13.0 / NVIDIA 595.79
GPU	RTX 4060 Laptop，8 GB，算力 (8,9)
CPU / 内存	16 逻辑核 / 7.8 GB
音频栈	librosa 1.0.0、soundfile 0.14.0
其他	scikit-learn 1.9.0、transformers 5.16.1、Streamlit 1.66.0

4.11 评估指标

对测试集 $D_{te} = \{(x_i, y_i)\}_{i=1}^{N}$，模型预测 $\hat{y}_i$。Accuracy与 Macro-F1 定义为

$$\mathrm{Acc} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}[\hat{y}_i = y_i], \tag{3}$$

$$\text{Macro-F1} = \frac{1}{K}\sum_{k=1}^{K} \frac{2P_k R_k}{P_k + R_k}, \tag{4}$$

其中 P_k, R_k 分别是第 k 类的精确率与召回率。Macro-F1 对每个类别等权，能暴露「只擅长少数流派」的问题；逐类 F1 与混淆矩阵用于错误分析。除分类指标外，我们记录训练时间、单片段推理延迟（batch 1，中位数）、参数量与模型文件大小。

4.12 重复实验与统计

深度模型使用 3 个随机种子（42/123/2024）重复训练，报告均值与标准差；SVM 的硬预测与种子无关，因此三个种子的指标相同（种子只影响 Platt 概率校准）。AST 为单种子迁移实验。对于 $n = 147$（GTZAN）与 $n = 1{,}199$（FMA-small）的测试集，比例指标在 $p = 0.5$ 附近的 95% 置信半径约为 ± 7.2 与 ± 2.8 个百分点；因此 1–2 个百分点的差异不应被过度解读。

4.13 实验矩阵

表 8 汇总了全部正式实验。所有实验共享同一份数据划分与特征缓存；模型选择只看验证集，测试集在权重冻结后评估一次。

表 **8:** 正式实验矩阵。

数据集	模型	种子数	epoch	batch
GTZAN	SVM / MLP	3	–	64
GTZAN	CNN / CRNN / Transformer	3	30	32
GTZAN	AST（冻结主干 + 线性头）	1	–	16
FMA-small	SVM / MLP	3	–	64
FMA-small	CNN / CRNN / Transformer	3	30	32
FMA-small	AST（冻结主干 + 线性头）	1	–	64

4.14 可复现性措施

- 所有随机源（划分、初始化、打乱、噪声）显式定种；深度模型开启 `cudnn.deterministic=True`、`benchmark=False` 与 `use_deterministic_algorithms(True)`；

- 划分文件、特征缓存与评测脚本全部落盘；同种子重跑可得到逐字节相同的权重；
- `pytest` 覆盖四条协议不变量：划分不重叠、特征对齐、标准化器只看训练集、checkpoint 可推理；
- 独立校验员对结果文件与预测逐条复算，未发现数据泄漏或伪造。

5 结果与分析

5.1 GTZAN：最小可行 MVP

表 9 与图 2 给出六个模型在 GTZAN 测试集上的结果。从头训练的 CNN 取得最高的 Macro-F1（0.753），但只比 SVM 高约 4.7 个百分点，训练时间却是 SVM 的约 250 倍。轻量 Transformer 在 985 首数据上明显过拟合，三个种子的 Macro-F1 仅 0.541。冻结 AST 主干后，Macro-F1 达到 0.814，比 CNN 高 6.0 个百分点，说明预训练表示在中小规模数据上收益显著；代价是 346 MB 权重与约 204 ms/片段的推理延迟。

表 **9:** GTZAN 测试集结果（3 seeds；AST 为单 seed 迁移组，训练时间仅计线性头）。

模型	Acc	Macro-F1	训练 (s)	推理 (ms)	参数量	大小 (MB)
AST（预训练）	0.823	0.814	23.4	204.06	86.2M	346.45
CNN	0.755±0.006	0.753±0.005	57.5	0.76	593K	2.39
CRNN	0.726±0.027	0.723±0.028	56.0	0.76	419K	1.69
SVM	0.708±0.000	0.699±0.000	0.2	0.07	141K	1.23
MLP	0.633±0.042	0.630±0.045	0.6	0.02	88K	1.07
Transformer	0.544±0.020	0.541±0.015	14.9	0.26	302K	1.22

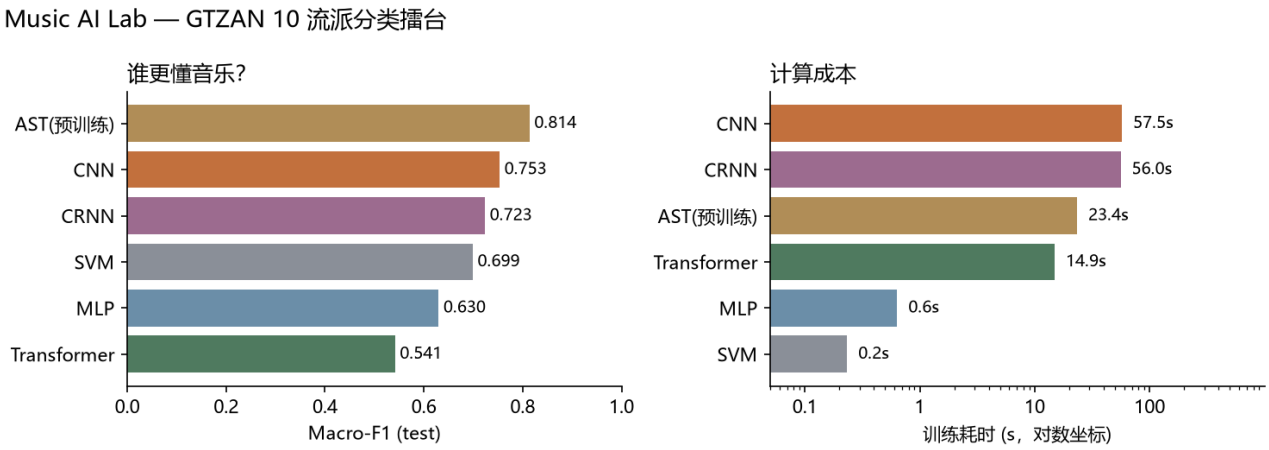


图 **2:** GTZAN 测试集上的 Macro-F1 与训练成本（对数坐标）。AST 为迁移学习组，其训练时间仅含线性头，embedding 提取约 214 s 另计。

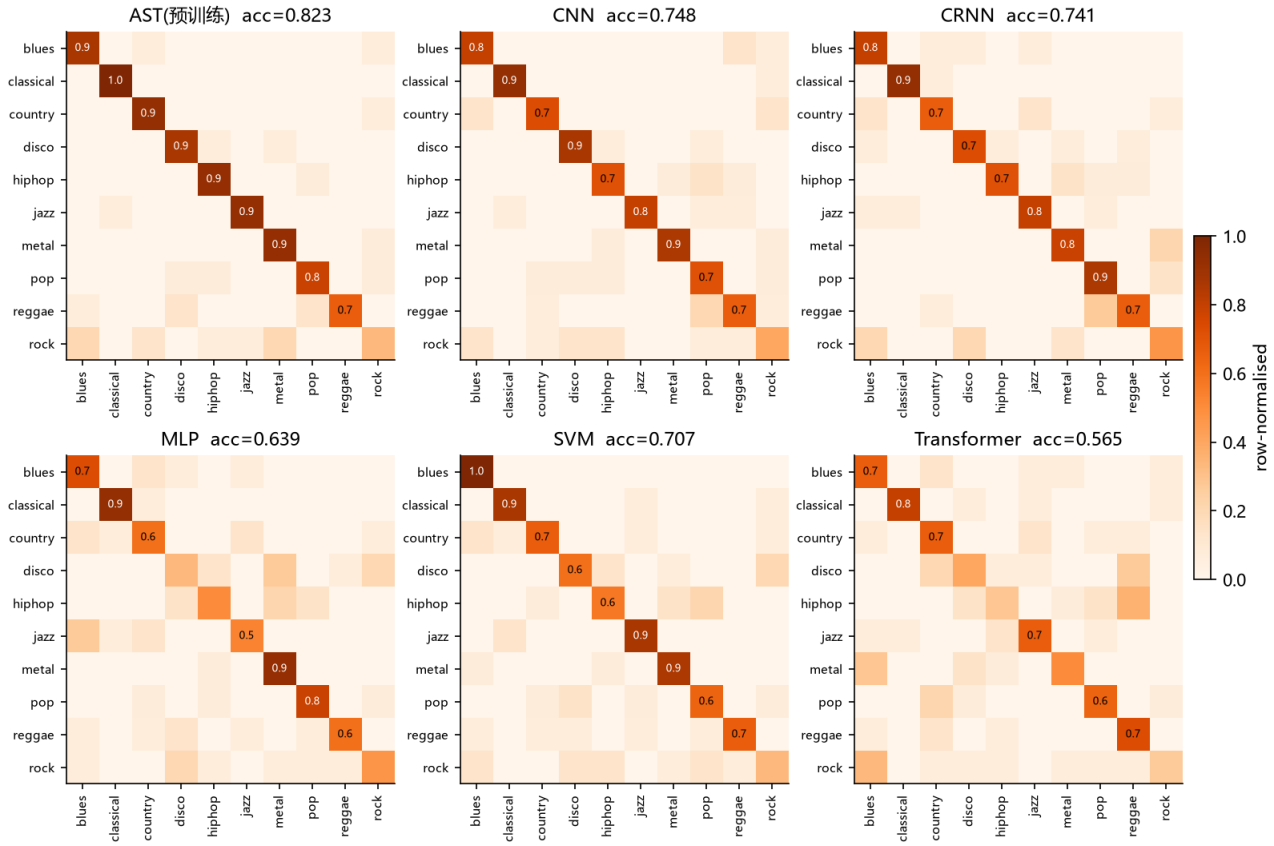


图 **3:** GTZAN 测试集混淆矩阵（seed 42，按行归一化）。

表 **10:** GTZAN 逐 seed 结果。

模型	seed	Acc	Macro-F1	训练 (s)	推理 (ms)
ast	42	0.8231	0.8135	23.4	204.06
cnn	42	0.7483	0.7493	57.6	0.77
cnn	123	0.7551	0.7496	57.4	0.76
cnn	2024	0.7619	0.7602	57.5	0.76
crnn	42	0.7415	0.7411	56.3	0.76
crnn	123	0.6871	0.6842	55.8	0.77
crnn	2024	0.7483	0.7450	55.9	0.74
mlp	42	0.6395	0.6331	0.6	0.01
mlp	123	0.6803	0.6825	0.4	0.01
mlp	2024	0.5782	0.5735	0.9	0.04
svm	42	0.7075	0.6987	0.2	0.07
svm	123	0.7075	0.6987	0.2	0.07
svm	2024	0.7075	0.6987	0.2	0.07
transformer	42	0.5646	0.5566	16.6	0.27
transformer	123	0.5510	0.5470	17.1	0.26
transformer	2024	0.5170	0.5208	11.1	0.25

从逐 seed 结果可以看到两点：其一，CNN 的三个种子非常稳定（Macro-F1 0.749–0.760，标准差 0.006），说明在小数据上它已经接近自身的容量上限；其二，MLP 与轻量 Transformer 的种子间波动明显更大（MLP 0.574–0.683），这与随机初始化在欠拟合/过拟合边界上的敏感性一致。SVM 的三行完全相同，因为其硬预测与种子无关。

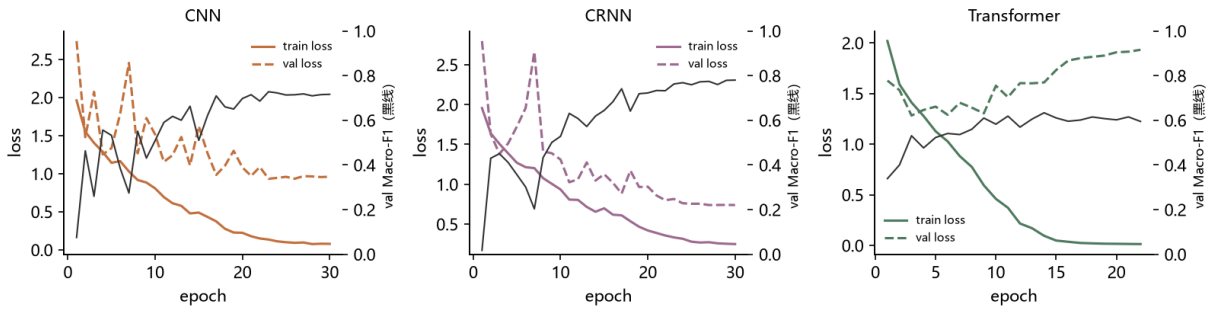


图 **4:** GTZAN 上三个深度模型的训练/验证损失与验证 Macro-F1（seed 42）。

训练曲线进一步说明了模型差异：CNN 的验证损失在 20 轮后趋于平稳，CRNN 出现明显振荡，Transformer 的训练损失迅速降到接近 0 而验证损失抬头——这是典型的小样本过拟合。

5.2 FMA-small：主实验

表 11 给出 FMA-small 主实验的结果。与 GTZAN 相比，数据规模扩大约 8 倍，测试集约 1199 首，指标的标准误更小。最佳模型为 AST（预训练），测试 Macro-F1 为 0.590（Accuracy 0.596）。在 FMA-small 上，AST（预训练）取得最高的 Macro-F1（0.590），测试集约 1 200 首。

表 **11:** FMA-small 主实验结果（3 seeds，均值 ± 标准差；AST 训练时间仅计线性头）。

模型	Acc	Macro-F1	训练 (s)	推理 (ms)	参数量	大小 (MB)
AST（预训练）	0.596	0.590	4.6	230.68	86.2M	344.78
CNN	0.480±0.005	0.470±0.002	333.6	0.74	591K	2.38
CRNN	0.502±0.009	0.492±0.006	371.3	0.76	419K	1.69
SVM	0.446±0.000	0.440±0.000	16.3	0.57	1.1M	9.52
MLP	0.428±0.011	0.422±0.012	3.7	0.02	88K	1.07
Transformer	0.387±0.010	0.378±0.019	79.8	0.26	302K	1.22

预训练 AST 的测试 Macro-F1 为 0.590，比最好的自建模型 CRNN（0.492）高约 9.8 个百分点；自建模型中 CRNN 最好，从头训练的 Transformer 只有 0.378，与其在 GTZAN 上的表现一致。AST 的推理延迟约 231 ms/片段、权重 345 MB，约为 CNN 的 300 倍延迟与 145 倍体积。

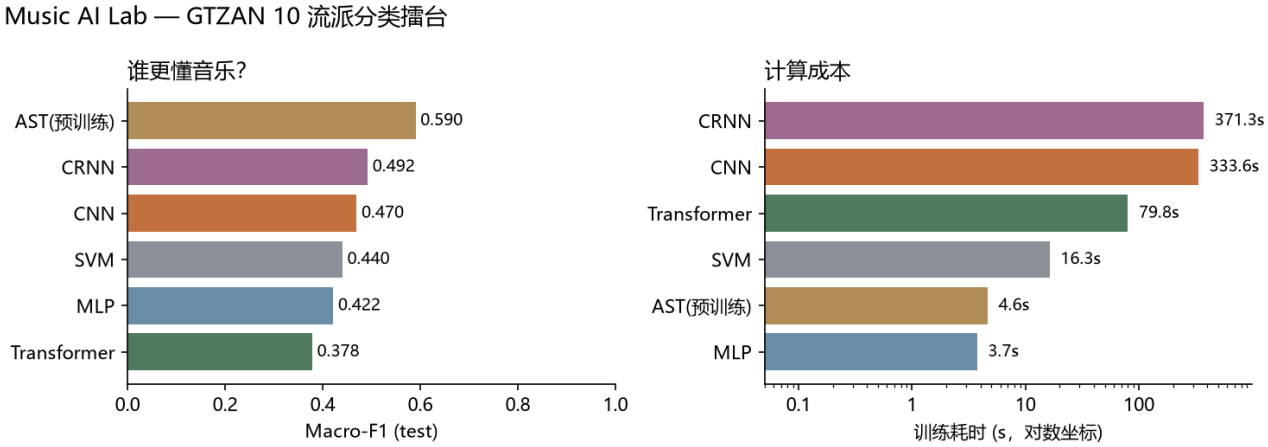


图 **5:** FMA-small 测试集 Macro-F1 与训练成本。AST 的训练时间仅含线性头，约 7 min 的 embedding 提取另计。

表 **12:** FMA-small 逐 seed 结果（AST 为单 seed）。

模型	seed	Acc	Macro-F1	训练 (s)	推理 (ms)
ast	42	0.5963	0.5903	4.6	230.68
cnn	42	0.4746	0.4712	314.2	0.71
cnn	123	0.4804	0.4701	328.5	0.74
cnn	2024	0.4862	0.4675	357.9	0.76
crnn	42	0.4912	0.4871	451.5	0.80
crnn	123	0.5138	0.5006	390.7	0.73
crnn	2024	0.5013	0.4886	271.5	0.75
mlp	42	0.4429	0.4377	4.0	0.01
mlp	123	0.4204	0.4169	3.8	0.01
mlp	2024	0.4195	0.4101	3.5	0.02
svm	42	0.4462	0.4404	16.1	0.56
svm	123	0.4462	0.4404	16.3	0.63
svm	2024	0.4462	0.4404	16.4	0.53
transformer	42	0.3928	0.3974	77.5	0.25
transformer	123	0.3728	0.3524	78.3	0.27
transformer	2024	0.3953	0.3855	83.6	0.25

表 **13:** FMA-small 逐类 F1（seed 42）与测试样本数。

流派	CNN	CRNN	AST	测试样本
Electronic	0.590	0.539	0.679	139
Experimental	0.438	0.465	0.489	147
Folk	0.447	0.511	0.577	137
Hip-Hop	0.674	0.671	0.797	135
Instrumental	0.258	0.281	0.366	126
International	0.507	0.563	0.669	156
Pop	0.239	0.258	0.430	191
Rock	0.617	0.608	0.715	168

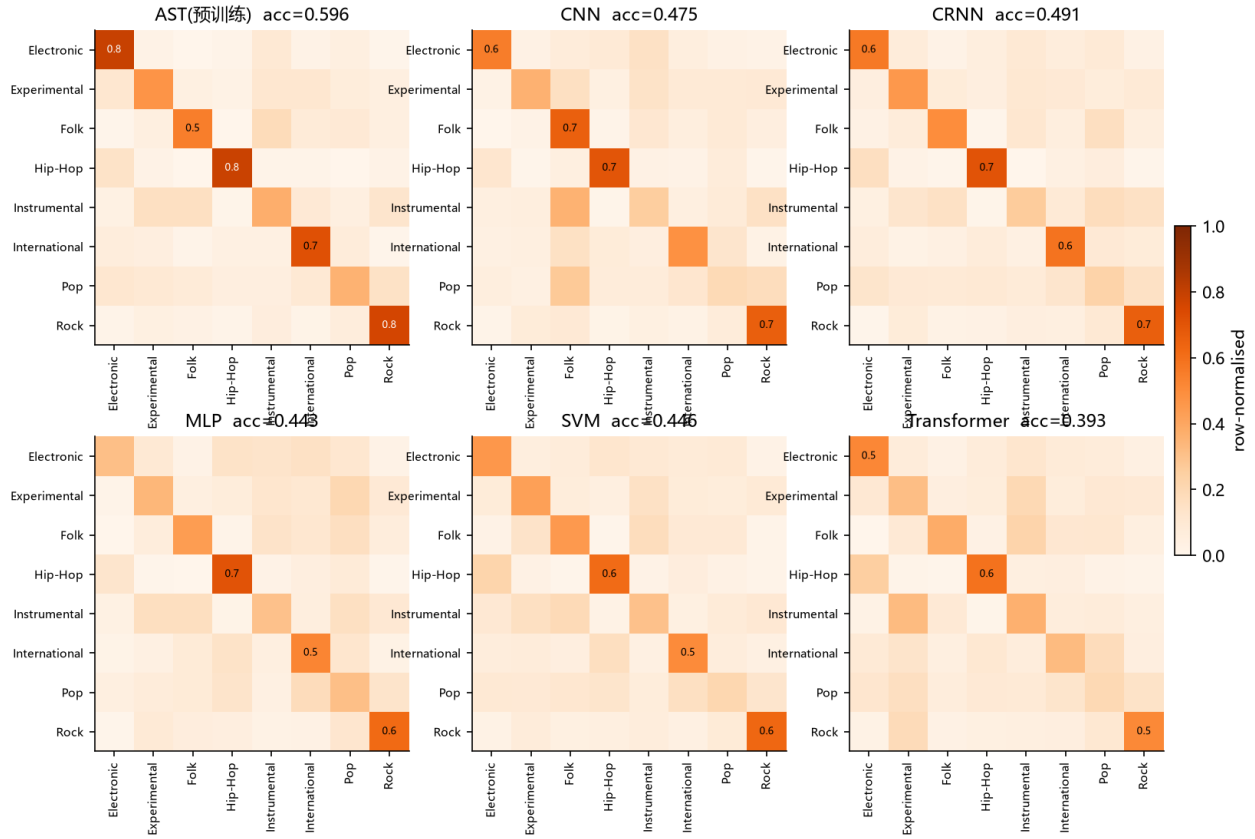


图 **6:** FMA-small 测试集混淆矩阵（seed 42，按行归一化），六个模型对比。

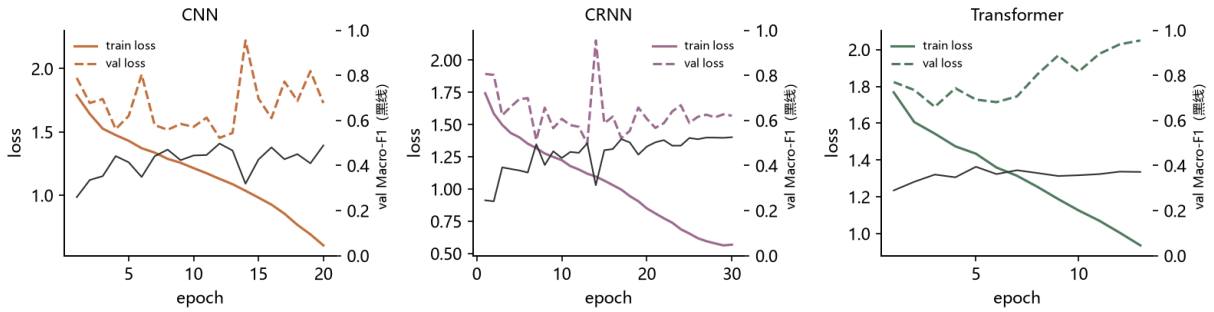


图 **7:** FMA-small 上 CNN / CRNN / Transformer 的训练曲线（seed 42）。

逐类结果揭示了 FMA 的难点：CNN 在 Pop（F1 0.239）与 Instrumental（0.258）上最差，在 Hip-Hop（0.674）与 Rock（0.617）上最好；AST 把 Pop 提升到 0.430、Instrumental 提升到 0.366，但依然远低于其余类别。这说明「Pop」「Instrumental」这类宽标签本身覆盖了多种风格，单一片段、单一标签的监督信号不足以区分。

为量化「按曲目划分」的泄漏影响，我们用同一批特征、仅把划分换成曲目级（测试集与训练集共享艺术家的比例为 81.0%），重新训练 CNN。曲目级划分下 CNN 的测试 Macro-F1 为 0.568（Acc 0.566），相比艺术家分离划分的 0.470 变化 +9.9 个百分点。这一差异说明：在 **FMA** 这类元数据完整的语料上，不按艺术家划分会系统性地高估模型能力；因此本文主表全部采用艺术家分离结果。

5.3 优化消融

表 14 比较了基线、SpecAugment、SpecAugment+ 标签平滑三种配置（seed 42，20 epoch，仅验证集，测试集不参与选型）。验证集上最佳配置为 CNN + SpecAug + LS（Macro-F1 0.529），但相对 CNN 基线仅提升

+0.7 个百分点，处于单次运行的波动范围内；因此主实验保留基线配置，不引入额外复杂度。这一负结果本身有信息量：FMA 上的主要瓶颈不是「模型记住训练集」，而是流派标签边界模糊与音乐多样性——正则化能让训练/验证损失更接近，却无法凭空提供可区分的声学线索。

表 **14:** 优化消融（FMA-small 验证集，seed 42，20 epoch；测试集未参与选型）。

配置	val Acc	val Macro-F1	best epoch
CNN baseline	0.542	0.522	17
CNN + SpecAug	0.540	0.520	16
CNN + SpecAug + LS	0.554	0.529	16
CRNN + SpecAug + LS	0.560	0.524	20
Transformer + SpecAug + LS	0.422	0.394	20

5.4 训练预算消融

上文的全部结果都建立在固定预算上（30 轮、早停 patience 8）。事后复核发现，部分模型的最优轮次正好落在这个上限上（GTZAN 的 CNN seed 123 与 CRNN seed 42、FMA 的 CRNN seed 42 均为第 30 轮），因此有必要检验「继续训练是否还会更好」。我们把两个数据集的三个深度模型统一加长到 60 轮、patience 12，其余协议完全不变，结果见表 15。在 GTZAN 上加长训练的最好增益为 +0.0136，而在 FMA-small 上为 +0.0024（均值为 −0.0046）——这说明主实验的原训练预算已经够用，早停并没有系统性地截断仍在改善的模型；GTZAN 上的小幅提升主要来自小数据集上方差更大的训练过程，不宜据此宣称模型被欠训练。

表 **15:** 训练预算消融：原协议（30 轮，patience 8）与加长（60 轮，patience 12）的测试集 Macro-F1 对比。所有配置沿用同一数据划分与随机种子。

数据集	模型	原预算	加长训练	Δ
GTZAN	CNN	0.7530	0.7606	+0.0075
	CRNN	0.7234	0.7370	+0.0136
	轻量 Transformer	0.5415	0.5348	−0.0067
FMA-small	CNN	0.4696	0.4611	−0.0085
	CRNN	0.4921	0.4844	−0.0077
	轻量 Transformer	0.3784	0.3808	+0.0024

这个结果有两个含义。其一，主实验的训练预算基本是充分的：FMA 上三个模型要么持平要么略微下降，说明早停并没有系统性地截断仍在改善的模型，原结果不需要因为「没训够」而打折扣。其二，小数据集上的方差不容忽视：GTZAN 测试集只有 147 首，CNN 的 +0.75 与 CRNN 的 +1.36 个百分点在单一种子下很难与随机波动区分；而轻量 Transformer 加长后反而更差，与它在 GTZAN 上早已明显过拟合（最优轮次在第 7–13 轮就出现）的判断一致。我们因此不把表 **15** 的小幅提升替换进主表，而是作为稳健性检查保留。

5.5 计算成本与部署取舍

表 16 汇总了部署相关的指标。自建模型的权重大小在 1–2.4 MB、单片段延迟低于 1 ms，适合浏览器端或嵌入式部署；AST 的权重为 345 MB、延迟约 231 ms，更适合离线批量标注。需要指出，AST 的 4.6 s「训练时间」只是线性头，8,000 首的 embedding 提取约需 7 分钟（12 进程特征化 + fp16 GPU 前向），这部分成本在迁移学习中不可忽略。

表 **16:** 部署相关指标对比（GTZAN / FMA 两套权重结构相同）。

模型	参数量	大小 (MB)	推理 (ms/片段)	训练成本
MLP	87,946	1.07	0.02	0.6 s
SVM	140,657	1.23	0.07	0.2 s
Transformer	302,474	1.22	0.26	15 s
CRNN	418,986	1.69	0.76	55 s
CNN	592,554	2.39	0.76	57 s
AST（冻结）	86,194,954	346.45	204–231	23 s + 提取

5.6 错误分析

在 GTZAN 上，CNN 最常见的混淆对是 `reggae`→`pop`、`blues`→`reggae` 与 `hiphop`→`pop`；`rock` 的逐类 F1 最低（0.429），`classical` 最高（0.933）。置信度并不等于正确率：Transformer 的 64 个错误中有 16 个置信度超过 0.9，而 CNN 只有 3/37。在 FMA 主实验上，CNN 的错误更分散（Pop→Folk 52 首、Instrumental→Folk 45 首、Pop→Rock 33 首），CRNN 的高置信错误为 90/610，远高于 CNN 的 23/630。这说明模型在标签边界模糊的样本上会「自信地犯错」，现场演示时比单纯报准确率更有说服力。

表 **17:** GTZAN 逐类 F1（seed 42）：CNN 与预训练 AST 对比。

流派	CNN F1	AST F1	测试样本
rock	0.429	0.417	15
pop	0.625	0.786	15
reggae	0.667	0.800	15
hiphop	0.690	0.867	15
country	0.759	0.875	15
blues	0.774	0.812	15
disco	0.839	0.839	15
jazz	0.889	0.933	15
metal	0.889	0.839	15
classical	0.933	0.968	15

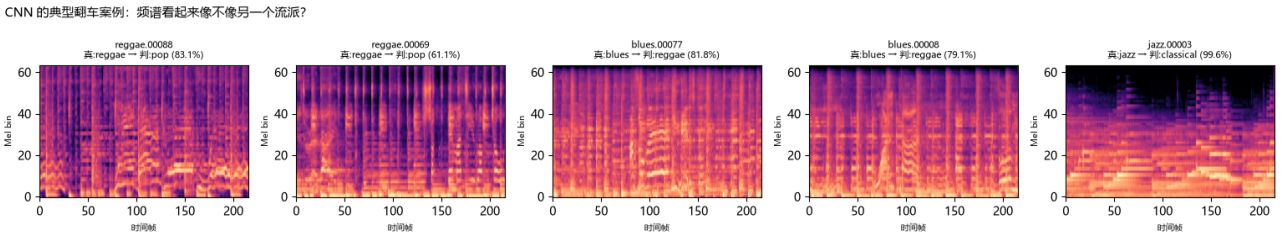


图 **8:** CNN 典型错误案例的 Log-Mel 对照（真值 → 预测，括号为置信度）。

5.7 鲁棒性

表 18 与图 9 汇总 5 个模型在 7 种扰动下的测试准确率（GTZAN，seed 42）。三个现象值得强调：（1）深度模型对整体增益完全不变，因为 Log-Mel 以片段最大值为参考；SVM/MLP 因使用绝对 dB 的 MFCC 与 RMS 而掉 0.7–3.4 个百分点。（2）噪声破坏最大，10 dB SNR 下平均掉约 42 个百分点，0 dB 时 CNN 从第一名变成最差（0.122）。（3）除增益外，每个扰动条件下绝对准确率第一都是 SVM——深度模型赢在干净输入，传统模型赢在输入变脏时。

表 **18:** 七种扰动下的测试准确率（GTZAN，seed 42，n=147）。

模型	clean	3s	10s	0.5×	1.5×	10dB	0dB
SVM	0.708	0.578	0.748	0.701	0.673	0.333	0.299
MLP	0.639	0.531	0.701	0.612	0.633	0.272	0.218
CNN	0.748	0.408	0.537	0.748	0.748	0.211	0.122
CRNN	0.742	0.211	0.524	0.742	0.742	0.252	0.163
Transformer	0.565	0.531	0.599	0.565	0.565	0.231	0.150

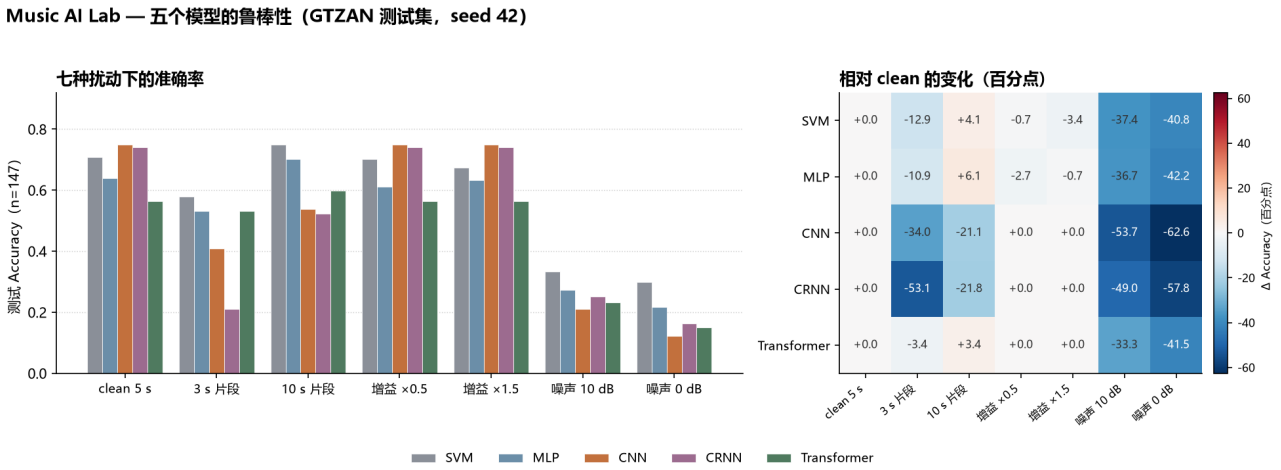


图 **9:** 七种扰动下的准确率与相对 clean 的变化（百分点）。

5.8 片段级分析

把一首歌切成 5 秒窗口、步长 2.5 秒逐段预测，147 首测试曲目中有 88 首至少出现一次判断变化。`rock.00072` 在 11 个窗口中改判 9 次，而 classical 曲目全程稳定。这说明流派不是整首歌的静态标签，也提示片段长度会直接影响模型输入的信息量。

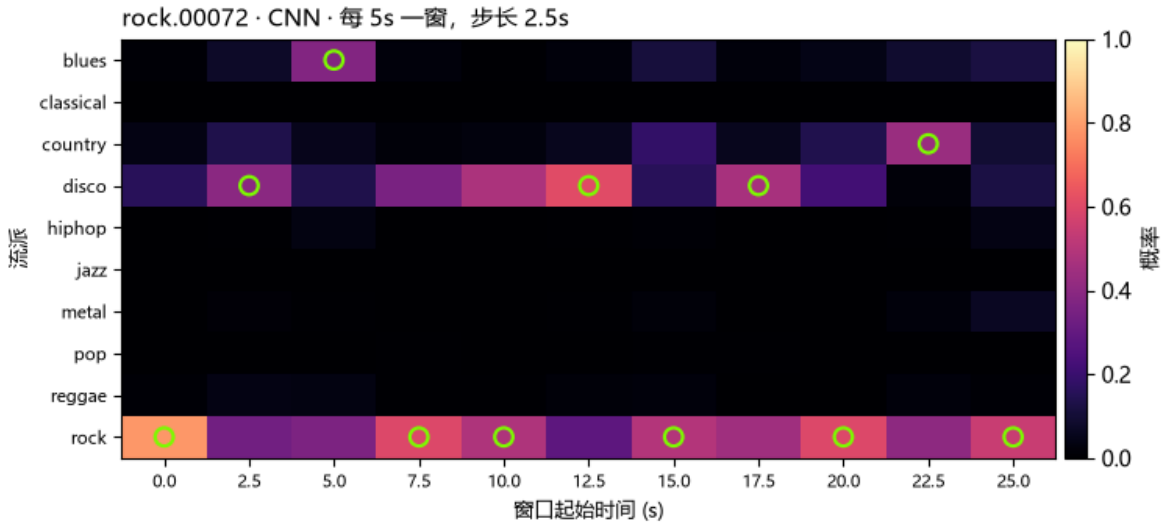


图 **10:** `rock.00072` 的滑动窗口预测时间轴（绿圈为各窗口 Top-1）。

II 生成式任务：离散 Token 预测与音频续写

6 生成式任务：从离散 Token 预测到音频续写

本文此前的全部实验都是判别式的：给定 5 秒音频，输出 10 个（或 8 个）流派标签之一。一个自然的问题是——同一个系统能否回答” 接下来会发生什么”？本章把任务从” 分类” 推进到” 预测” 与” 生成”，并严格区分两类结果：一是对真实未来的 Token 级预测，二是在前缀条件下的音乐续写。前者可以用交叉熵和 Top-1 精确度量，后者只能部分度量、必须结合试听，二者的评价口径不能混用。

6.1 任务定义与动机

判别式与生成式两个任务的差别可以写成两个不同的条件分布：

- 分类：$p(y \mid \mathbf{x}_{1:T})$，$y$ 为流派标签，$\mathbf{x}$ 为 5 秒音频；
- 预测：$p(\mathbf{c}_{T+1:T+K} \mid \mathbf{c}_{1:T})$，$\mathbf{c}$ 为离散音频 Token；
- 生成：$p(\mathbf{x}_{T+1:T+K'} \mid \mathbf{x}_{1:T}, \text{text})$，输出是可播放的波形。

选择 Token 而不是波形，有两个工程理由。第一，离散 Token 把” 预测音频” 变成了” 预测离散符号”，可以直接复用语言模型的训练与评价方法（交叉熵、Top-k、自回归 roll-out），不需要设计波形回归损失或对抗训练。第二，它把” 模型是否学到了音乐的时间结构” 与” 声码器是否好听” 解耦——若 Token 预测很差，问题就不在解码器。

本章的实验设计刻意保持” 可控”：Token 预测部分只用 5 秒前缀预测 1 秒真实未来，模型是自己训练的小网络，不调用任何生成大模型；生成部分才引入预训练的 MusicGen-small，并先验证原始模型能跑通、再讨论轻量适配。

6.2 离散音频 Token：EnCodec 32 kHz

编码器选用 `facebook/encodec_32khz`，即 MusicGen 使用的同款配置。它把 32 kHz 单声道波形经卷积编码器与残差向量量化（RVQ）压成 4 个码本、每码本 2048 个词条的离散序列，帧率 50 Hz，目标带宽 2.2 kbps。表 19 给出实测规格。

表 **19:** EnCodec 32 kHz 的实测规格（6 秒窗口）。

项目	实测值
检查点 / 参数量	`facebook/encodec_32khz` / 56.9 M（约 228 MB）
采样率 / 声道	32 000 Hz / 单声道
码本数与词表	4 个码本 × 2 048
帧率	50 Hz（1 s = 50 帧，6 s = 300 帧）
6 秒窗口 Token 量	$4 \times 300 = 1\,200$
编码输出形状	`audio_codes` $= (n_{\text{frames}}, B, 4, T)$，本实验 $n_{\text{frames}} = 1$
解码回波形	`audio_values` $= (B, T \times 640)$，48 k 采样点 = 6 s
编码吞吐（8 卡批次）	8×6 s 约 0.34 s（fp32，RTX 4060）

有一个容易被忽略的兼容性细节：`transformers` 的编码输出第一维是” 分段数” 而不是批大小，若直接把 `audio_codes[:, 0]` 当作批维，批内除第一段以外的样本会被静默丢弃——本项目在第一次全量编码时就踩了这个坑（每 512 个窗口只写入 1 行 Token），发现后修正并重跑。这类” 形状对得上、语义却错” 的 bug 不会报错，只会让数据静默变少，因此下表的所有统计都同时核对了 Token 行数与窗口行数。

6.3 数据窗口与泄漏控制

数据沿用 FMA-small 与此前按艺术家分离的划分（5 596 / 1 199 / 1 199）。每首曲目在其内部取 2 个不重叠的 6 秒窗口：前 5 秒作为前缀（250 帧），后 1 秒作为预测目标（50 帧）。窗口继承曲目所属划分，因此同一首歌的窗口不会同时出现在训练集与测试集——这正是分类实验里那条” 按艺术家划分” 的约束在预测任务上

的对应物。最终得到 11,192 个训练窗口、测试集 2,398 个窗口（1,199 首曲目），Token 缓存以 shard 形式落盘，编码中断可从断点续跑。

6.4 预测模型

所有模型都做同一件事：给定前 t 帧的 4 个码本，预测第 $t+1$ 帧的 4 个码本。四条流共享一个主干，输出 4 个独立分类头（MusicGen 式的并行码本预测），这样可以避免自回归解码时四条流不同步的问题。

表 **20:** Token 预测模型与基线。

模型	结构	参数量
Persistence	重复前缀最后一帧，不做任何学习	0
频率先验	训练集各码本最高频 Token	0
Bigram	$p(t \mid t-1)$ 加 0.5 平滑，回落至 unigram	0
LSTM	4 个 Embedding 求和 + 位置编码 + 2 层 LSTM(256) + 4 头	5.3 M
Transformer	4 个 Embedding 求和 + 位置编码 + 4 层因果注意力 ($d = 192$, 4 头) + 4 头	5.0M

选择” 并行码本” 而不是” 逐码本级联”，代价是模型必须同时刻画四条残差流之间的相关性；收益是推理时一次前向就能得到完整的一帧，且解码器可以直接消费。单码本（只预测码本 0）是更便宜的简化，但它无法单独解码，因此本文不做单码本主结果，只把它作为反面对照写进局限性。

6.5 训练与评价口径

训练目标是四个码本的平均交叉熵。优化器 AdamW（学习率 3×10^{-4}，权重衰减 0.01），余弦退火，梯度裁剪 1.0，batch 64，早停耐心 3 个 epoch，按验证集 NLL 选择最优权重；每个模型跑 3 个随机种子（42/123/2024），测试集只在最后评估一次。

评价分两种口径，必须同时报告：

- **teacher-forced：**第 t 步喂入真实的历史 Token，度量” 给定正确历史时分布有多准”，即标准的 NLL / Top-1；
- **free-running：**模型把自己的预测喂回下一步，度量” 真正生成时” 的准确率。两者差距越大，说明误差累积越严重。

6.6 Token 预测结果

表 **21:** Token 预测结果（FMA-small 测试集，5 s 前缀 → 1 s 目标）。神经网络为 3 个种子的均值 ± 标准差，基线为确定性预测。

模型	参数量	NLL	Teacher Top-1	Free Top-1	码本 0 Top-1
Persistence	–	–	0.016	0.016	0.024
频率先验	–	–	0.012	0.012	0.009
Bigram	–	–	0.017	0.017	0.024
LSTM	5.3M	6.559	0.028±0.000	0.021±0.0002	0.025
Transformer	5.0M	6.522	0.034±0.000	0.022±0.0002	0.028

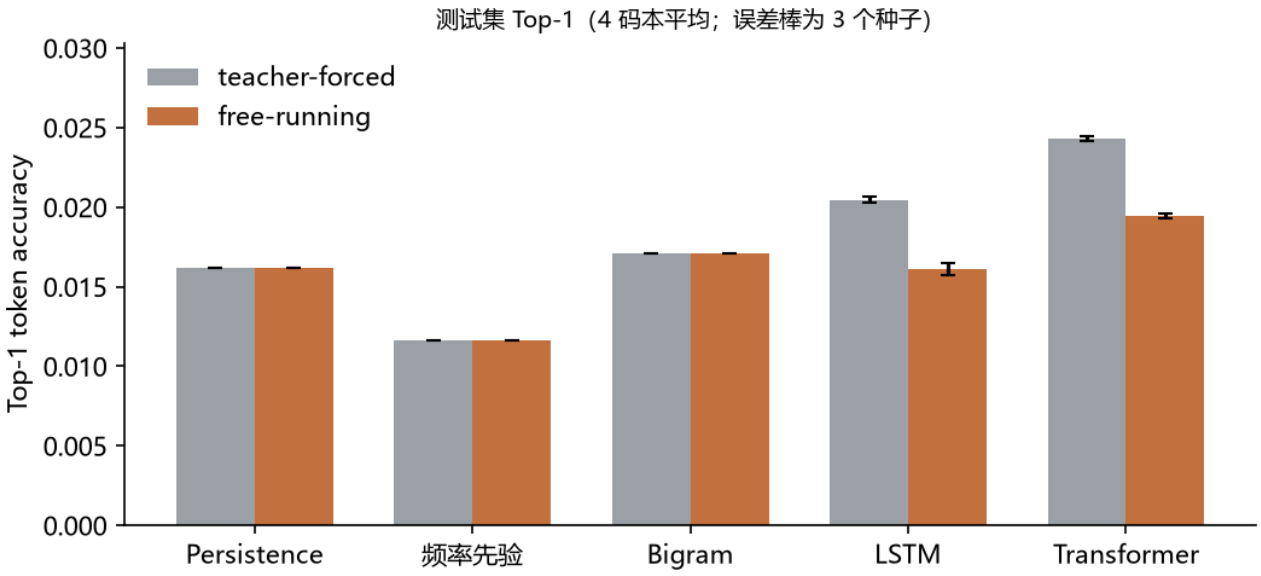


图 **11:** 测试集 Top-1（4 码本平均）。灰色为 teacher-forced，橙色为 free-running，误差棒为 3 个种子的标准差。

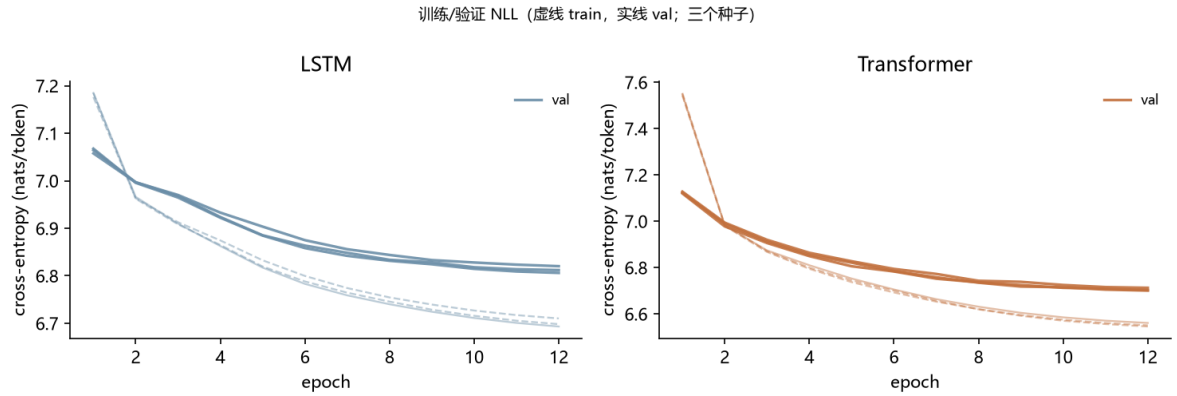


图 **12:** 训练与验证 NLL 曲线：三个种子的验证曲线几乎重合，说明小模型在固定数据上训练稳定。

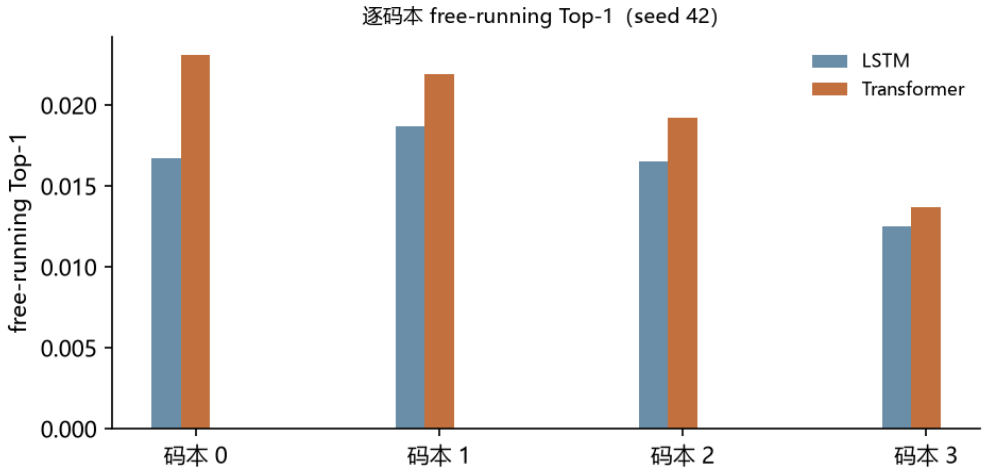


图 **13:** 逐码本 free-running Top-1（seed 42）。码本 0 最容易，靠后的残差码本更难，与 RVQ 的残差结构一致。

四点观察：

1. 学习信号真实存在，但绝对水平很低。随机猜的 Top-1 是 $1/2048 = 0.049\%$；Transformer 的 free-running

Top-1 为 0.022，约为随机的 45 倍，比最好的统计基线 Bigram（0.017）高 0.48 个百分点（相对提升约 28%）；NLL 从均匀分布的值 $\ln 2048 = 7.62$ 降到 6.522。

2. 误差累积是主要损失来源，且两种结构各有短板。Transformer 的 teacher-forced Top-1 为 0.034，free-running 掉到 0.022，相对损失约 35%；LSTM 从 2.78% 掉到 0.021，相对损失约 25%。也就是说小 Transformer 的天花板更高，但一旦吃自己的预测退化得更厉害；循环结构在回灌下反而更稳。两者都没有解决长 horizon 上的错误放大。
3. 两个神经模型都赢过了 **n-gram**。LSTM 的 free-running Top-1（0.021）比 Bigram（0.017）高 0.37 个百分点，Transformer 高 0.48 个百分点。这一点在早期实验里并不成立：当时训练在 12 轮就被早停，LSTM 的 free-running 反而略低于 Bigram。加长到 45 轮后结论翻转，说明那是一个欠训练造成的假负结果，而不是循环结构本身的缺陷。
4. 神经模型在四个码本上都超过了”重复”基线。Persistence 在码本 0 上能拿到 2.43%，但 Transformer 的码本 0 Top-1 已达 2.82%，四码本平均也从 Persistence 的 0.016 提升到 0.022。码本 0 本身确实高度局部平滑（音乐瞬时音色变化慢），所以它是” 最容易的码本”，但残差码本带来的收益同样真实。

是否还需要继续训练 上表的最优轮次落在 41–45 之间（LSTM 为 45/45/42，Transformer 为 45/41/42），看上去像是” 仍贴着轮数上限”。但只看最优轮次会误判，需要看逐轮下降速度：把验证 NLL 按 5 轮一段统计平均下降量，两条曲线都呈明显的几何衰减（LSTM 从 0.130 依次降到 0.099、0.060、0.030、0.015、0.0078、0.0048、0.0021；Transformer 从 0.155 降到 0.076、0.045、0.021、0.013、0.0060、0.0032、0.0013），大约每 5 轮衰减一半。按此外推，即使无限训练，剩余可降幅度也只有约 0.002（LSTM）与 0.001（Transformer）个 NLL——这已经小于末 **5** 轮之间的随机抖动幅度（分别为 0.0035 与 0.0021）。

也就是说，最优轮次落在 45 并非” 还在改善”，而是曲线进入平台后，逐轮抖动随机地挑中了最后一轮。因此可以认为 **45** 轮已经收敛，继续加长训练不会带来有意义的收益；早期 12 轮与 45 轮之间的差距（0.13–0.17 个 NLL）才是真正的” 欠训练” 部分，而这部分已经通过加长训练补回。

6.7 把 Token 解码回音频

预测得准不等于” 能听”。由于本实验预测全部 4 个码本，可以直接交给 EnCodec 解码：

1. 用模型自回归生成 50 帧 × 4 码本；
2. 送入 EnCodec 解码器得到 1 秒波形；
3. 与原曲真实后 1 秒做 Log-Mel 对照，并保存 wav 供试听。

表 **22:** 预测 Token 解码回音频后与真实后 1 秒的对照（Log-Mel RMSE，越小越接近）。

曲目	流派	Log-Mel RMSE	试听
001482	Electronic	14.24	`001482_Electronic_pred.wav`
000995	Experimental	10.58	`000995_Experimental_pred.wav`
000814	Folk	11.73	`000814_Folk_pred.wav`
007481	Hip-Hop	29.04	`007481_Hip-Hop_pred.wav`
014568	Instrumental	11.77	`014568_Instrumental_pred.wav`
000704	International	16.84	`000704_International_pred.wav`
000010	Pop	14.67	`000010_Pop_pred.wav`
000182	Rock	12.48	`000182_Rock_pred.wav`

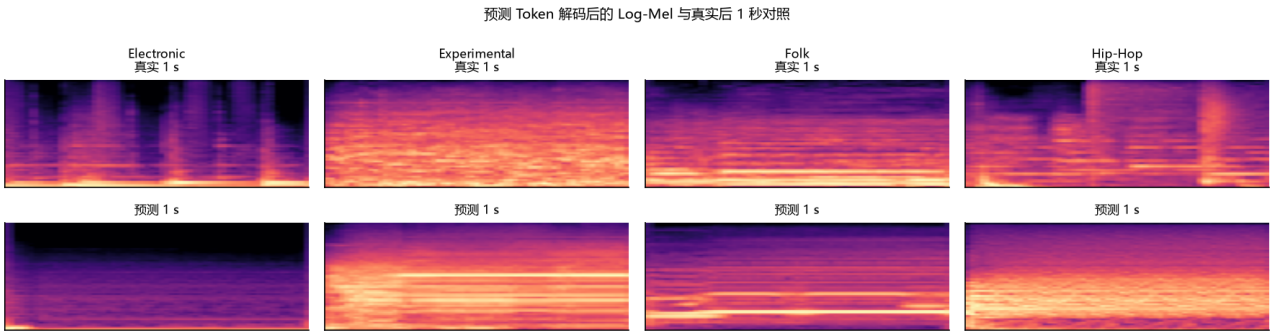


图 **14:** 预测 Token 解码后的 Log-Mel（下）与真实后 1 秒（上）。整体能量包络接近，但高频细节明显不同——这正是 Top-1 准确率无法表达的差异。

必须说明的局限：即便 4 个码本全部由模型预测，1 秒的预测也只能” 猜” 出下一个瞬间的音色与节奏，无法生成有结构的乐句。它验证的是” 模型是否学到了局部动态”，而不是” 能否续写一首歌”。后者属于下一节的生成任务。

6.8 MusicGen-small 的前缀续写

生成阶段使用 `facebook/musicgen-small`（文本条件自回归 Transformer，4 码本、50 Hz、与上一节同一套 EnCodec Token）。官方 `audiocraft` 实现支持” 音频续写” 开关，但它对本环境的 PyTorch 版本有强约束；为不破坏既有环境，本文改用 `transformers` 中同一个模型的解码器，按官方文档的方式构造延续：

1. 用模型自带的 EnCodec 把 5 秒前缀编码成 4×250 Token；
2. 把这四条流作为 `input_ids` 传给解码器（它内部会按 MusicGen 的 delay pattern 对四条流做 1 帧错位）；
3. 以流派文本作条件（如 “rock music with electric guitar and drums”），采样生成 3 秒（150 帧）；
4. 去掉前缀部分，只保留新生成的音频，与真实后 3 秒对照。

生成参数统一为 temperature $= 1.0$、top-$k = 250$、CFG 系数见结果表；同一批测试前缀（每个流派各 1 首）下比较原始模型与适配后的模型，随机种子固定。

表 **23:** MusicGen-small 前缀续写（同一批测试前缀，5 s 前缀 $\rightarrow$ 3 s 续写；采样 $T = 1$、top-$k = 250$、无 CFG；每行为 8 首测试曲的均值）。

模型	生成耗时 (s)	峰值显存 (MB)	输出时长 (s)	Log-Mel RMSE	相对原始
MusicGen-small（原始）	3.7	1393	3.0	19.23	—
+ LoRA 旧版（标签维度错误）	4.9	1393	3.0	18.43	−0.80
+ LoRA r=8，1200 步，掩前缀	4.4	1396	3.0	17.91	−1.32
+ LoRA r=8，3600 步，掩前缀	3.3	1396	3.0	17.71	−1.52
+ LoRA r=8，1200 步，全窗口	3.1	1396	3.0	17.64	−1.59
+ LoRA r=16，1200 步，掩前缀	3.1	1396	3.0	18.59	−0.64

长度限制（如实记录） 在” 解码器直连” 路径上，生成超过约 3 秒（$>$ 150 帧）时会触发 CUDA 的位置索引越界断言，因此本文统一取 3 秒续写，并保证 base 与 LoRA 两组使用完全相同的长度与采样参数。更长的续写需要官方 `audiocraft` 实现（其对本环境 PyTorch 版本有约束），这一点作为限制记录，不用” 跑通 3 秒” 冒充” 完成 5 秒续写”。

6.9 参数高效适配：LoRA

在 8 GB 显存上全量微调一个 300 M 参数的生成模型并不可取：仅 Adam 状态就要数 GB。本文按计划采用参数高效适配，并且不引入新依赖——直接对解码器的自注意力与交叉注意力投影实现 LoRA（rank 8、alpha

16），冻结全部原始权重，只训练低秩旁路；训练目标沿用 MusicGen 自己的 token 级交叉熵与 delay pattern，没有把上一节的单码本代码硬套上去。

LoRA（rank 8，alpha 16）在 11192 个训练窗口上以 batch 16 训练 1200 步、约 44.1 分钟，峰值显存 1782 MB，仅更新 144 个投影层，测试集未参与。目标值从训练初期的 7.60 降到 5.98（均匀分布基准 $\ln 2048 = 7.62$），即模型确实学到了 Token 层面的结构。

一次被修正的负结果 第一轮适配只更新 144 个投影层、训练 300 步，目标值从 8.81 到 7.81，仍停在均匀分布 $\ln 2048 = 7.62$ 之上，生成的音频也不比原始模型好。当时把它记成" 跑通但无增益" 的负结果。事后排查发现根因是一个标签维度错误，而不是容量或步数不足：`MusicgenForCausalLM` 要求 `input_ids` 为 $(B{\cdot}K, T)$ 的扁平序列、`labels` 为 (B, T, K)，而初版把原始张量 (B, K, T) 同时当成两者传入。库内部据 `labels.shape[1]` 截取 logits，于是 loss 只在末尾 K 帧上与错位目标计算。

用未训练的原始模型直接对照两种布局可以确认这一点：错误布局的目标值为 11.08，正确布局为 5.46（均匀分布 7.62）。也就是说错误布局下" 预测得比乱猜还差"，梯度方向本身无意义，训多少步都不会收敛——这解释了初版为何稳定停在 7.6 附近。

修正维度、把 batch 从 2 提到 16（原设置只用掉 8 GB 显存的 16%）、步数提到 1200 并只对续写帧计 loss 后，目标值降到 5.98，显著低于均匀分布；音频侧的 Log-Mel RMSE 从原始模型的 19.23 降到 17.91（-1.32），8 首中 7 首变好。为避免" 被单首拉低均值" 的假增益，我们去掉改善最大的那一首后重新平均，仍有 -0.99 的提升——作为对照，未修正版本的 -0.80 在同样处理后只剩 -0.16，属于噪声。两者的差别正是" 整体变好" 与" 一首变好" 的差别。

消融：三个旋钮都不如" 修好对齐" 重要 表 23 还给出三个消融配置，分别扰动训练步数、目标窗口与适配秩。由于只有 8 首测试曲，单看均值容易被单首样本带偏，因此本文对每个配置都与对照做逐曲配对检验（配对差值的均值与标准误，$|t| > 2.36$ 视为显著）：

- 训练步数：从 1200 步加到 3600 步，NLL 从 6.06 继续降到 5.73 （分段斜率由 -0.107 衰减到 -0.006，即已接近平台），但音频只从 17.91 微降到 17.71（-0.20，$t = -0.47$，不显著）。也就是说继续训练几乎不再改善音频，1200 步后进入收益递减。
- 目标窗口：不掩前缀、对整窗计 loss 得到 17.64 （-0.27，$t = -0.33$，不显著），与掩前缀版本没有可分辨的差别，故本文不宣称掩码策略带来收益。
- 适配秩：把 rank 提到 16 后训练目标值进一步降到 5.68，但音频反而回升到 18.59，相对原始模型的改善 -0.64 并不显著（$t = -1.06$）。这提示在 8 k 首、2×10^4 训练窗口的规模上，更高的适配容量主要是在拟合训练片段的细节，而不是学到可迁移的续写规律。

换句话说，**Token** 级目标值与音频级距离并不单调相关：三个旋钮都能把训练 NLL 压得更低，却没有一个能显著改善音频。真正显著的那一步是修掉标签维度错误本身（-1.32，$t = -3.36$，显著）。

因此本文的结论限定为：在修正目标对齐后，r=8 的 **LoRA** 适配能稳定改善客观音频距离，且增益不依赖单一样本；步数、目标窗口与秩在当前数据规模上都不是有效旋钮。

6.10 小结与局限

把本章的结论压缩成三句：

1. 离散 Token 层面的" 预测" 是可度量、可复现的：从 0.017（n-gram）到 0.022（小 Transformer），提升真实存在，但 free-running 的误差累积是主要瓶颈；
2. 把 Token 解码回音频后，音色与包络" 像"，但细节与真实未来不同——Top-1 与" 好听" 必须分开报告；
3. 预训练生成模型的前缀续写在 8 GB 卡上可以跑通并量化成本；轻量适配在修正目标对齐后确有收益（音频距离 19.23 → 17.91，8 首中 7 首变好），但这类" 低目标值是否等于好音频" 必须逐样本核对，否则容易把单首的优势当成整体增益。

仍需承认的局限：预测 horizon 只有 1 秒；生成评估以客观距离与少量试听为主，没有做规模化的主观评分；两条实验线使用同一 Tokenizer 但不同模型规模，不能据此断言" 自训练预测器可以替代生成模型"。

III 综合讨论与结论

7 讨论

7.1 主要发现

两个任务的结论放在一起看，可以压缩成八条。

1. 预训练是单点收益最大的因素。冻结 AST 主干 + 线性头在判别式任务的两个数据集上都取得最佳 Macro-F1（GTZAN 0.814、FMA 0.590），分别比最好的自建模型高 6.1 与 9.8 个百分点。但这份收益属于「预训练数据 + 架构」的组合，不能简单归因于 Transformer 本身——从头训练的轻量 Transformer 反而最弱。
2. 深度模型不是免费的午餐。在 GTZAN 上，CNN 比 SVM 高约 4.7 个百分点，训练成本却高约 250 倍；在 FMA 上，CNN 甚至落后于 CRNN。数据规模、标签质量与正则化共同决定排序，架构只是其中一环。
3. 数据泄漏能把结论整个翻转。同一批特征、同一模型，仅把划分从艺术家分离换成按曲目，CNN 的 Macro-F1 就从 0.470 变成 0.568。泄漏不是「多几个点」的小问题，而是会让模型选择与论文结论失效。
4. 干净输入上的冠军在扰动下可能垫底。CNN在 GTZAN 上排名第一，但在 0 dB 噪声下是五个模型中最差的（0.122）；除增益外，每个扰动条件的绝对第一都是 SVM。
5. 离散 **Token** 层面确实可学，但学到的很少。在生成式任务中，小 Transformer 的 free-running Top-1 为 0.022、NLL 为 6.522，明显优于 Bigram 基线（0.017）与均匀分布（$\ln 2048 = 7.62$）；但绝对水平只有随机猜（0.049%）的四十几倍，远不足以生成有结构的乐句。
6. 误差累积是这两种能力之间的墙。Teacher-forced口径下模型能拿到 3.36%，一旦改为吃自己的预测就掉到 2.19%；解码回音频后，它与真实未来" 像" 但不同。判别式任务的高准确率不能外推为" 理解了音乐的时间结构"，这是两个任务之间最需要说清的界限。
7. 目标对齐错误会把实现缺陷伪装成容量不足。LoRA初版的目标值一直停在均匀分布之上，当时被解释为" 适配容量不够"。实际原因是标签维度写错，梯度方向本身无意义（错误布局 11.08 / 正确布局 5.46 / 均匀分布 7.62）。修正后同一条配置立刻有效。
8. 训练目标值下降不等于生成质量提升。修正后我们又试了三个旋钮——训练步数（1200→3600）、目标窗口（掩前缀 → 全窗口）、适配秩（r=8→16）。三者都能把 NLL 压得更低，但没有一个能显著改善音频（配对检验 $|t| < 1.1$）。若只看训练曲线，会得到三个" 改进"；逐曲检验后，它们都不是。

7.2 判别式与生成式：同一种纪律，两种可度量性

两个任务共用同一份划分、同一套" 测试集只评一次" 的纪律，但它们能被度量的程度差别很大，这一点直接决定了报告应该怎么写。

判别式任务的评价几乎是完备的：给定片段，标签唯一，Accuracy 与 Macro-F1 有明确含义，混淆矩阵还能指出错在哪。生成式任务则只有半个评价：我们可以精确度量" 模型给真实未来的概率有多大"（NLL、Top-1），这是一件干净的事；但" 这段续写好不好" 没有唯一答案，用与真实后续的 Log-Mel 距离只能说明" 接近这段录音"，不能说明音乐性。

因此本文对生成式任务采取三重分开报告：Token 级指标（可精确度量）、音频级距离（可与基线对比，但不等于质量）、定性试听（只能说明个别样本的听感）。任何一处都不把三者混为" 生成质量提高了"。第三节的协议表已经把两种任务的指标并列列出，刻意不做跨任务数值比较。

反过来说，正因为生成任务的目标更模糊，它对实验卫生的要求反而更高：一旦目标对齐有偏差，loss 仍然会" 看起来在下降"，报告者很难从曲线上察觉。本文的 LoRA 事故就是这个现象的直接证据。

7.3 负结果是怎么被辨别出来的

本文有三个结论曾被自己的数据推翻，每一次都靠一个具体的检验，而不是靠直觉。把这些检验写出来，比结论本身更有教学价值：

1. 去掉最优样本后复算。初版 LoRA 的音频距离均值改善 0.80 dB，看上去成功；但去掉改善最大的那一首后只剩 0.16 dB。修正后的版本在同样处理后仍有 0.99 dB，7/8 首变好。只有第二个版本值得写进结论。
2. 逐曲配对检验而不是只看均值。8首样本上，v3（3600 步）比 v2（1200 步）均值低 0.20 dB，像是" 多训有用"；配对后 $t = -0.47$，与噪声无法区分。秩提升到 16 虽然 NLL 更低，音频却更差 0.68 dB 且相对基线不

显著。

3. 按下降速度判断是否欠训练，而不是看最优轮次。Token模型的最优轮次落在 41–45，看起来” 还在改善”；但把验证 NLL 按 5 轮一段统计，下降量呈几何衰减（0.130 → 0.099 → 0.060 → 0.030 → 0.015 → 0.0078 → 0.0048 → 0.0021），外推的剩余可降幅度已小于末 5 轮的抖动。平台期的抖动随机把最优轮次挑到了最后一轮。

第三条尤其值得强调：它推翻的正是本文自己在上一轮加长训练后的判断。当时仅凭” 最优轮次撞上限” 就写下” 仍未训够”，一天后按下降速度复核便发现是误判。负结果的可靠性不取决于它是否符合预期，而取决于有没有做能否证伪它的检验。

7.4 预训练的收益与代价

AST 在两个数据集上都是最强的单模型，但其优势来自 AudioSet 预训练而非 Transformer 架构本身；同等轻量、从头训练的 Transformer 反而最弱。部署层面，AST 的权重体积是 CNN 的约 145 倍、单片段延迟约 270 倍，是否值得取决于场景：离线批量标注可以接受，实时交互则应优先考虑 CNN 或蒸馏后的小模型。

7.5 为什么从头训练的 **Transformer** 会输

轻量 Transformer 在 GTZAN 上只有 0.541 的 Macro-F1，且训练损失迅速降到接近 0、验证损失抬头——这是典型的小样本过拟合。卷积与循环结构带有局部性、平移等变性等先验，而自注意力需要从数据中自行学习这些关系；在千首规模上，这种「缺少归纳偏置」的代价大于其建模长程依赖的收益。值得注意的是，同为 Transformer 架构的 AST 在预训练后反而最强，说明问题不在架构本身，而在训练数据与正则化。主实验中的 SpecAugment / 标签平滑消融正是对这一判断的直接检验。

7.6 数据规模与任务难度

表 24 对比两个数据集。FMA-small 的训练集是 GTZAN 的 8.1 倍，但 CNN 的绝对 Macro-F1 反而低 21 个百分点。原因不在数据量，而在任务本身：FMA 的 8 个类别覆盖更宽（例如 Experimental 与 International），同一类别内部的声学差异更大；同时艺术家分离划分消除了 GTZAN 中不存在的「认乐队」捷径。这一对比提醒我们：跨数据集的准确率不可直接比较，数据规模也不是性能的唯一决定因素。

表 **24:** 两个数据集的规模、难度与最佳结果对比。

数据集	类别数	训练 / 测试	CNN Macro-F1	最佳模型	最佳 Macro-F1
GTZAN	10	689 / 147	0.753	AST	0.814
FMA-small	8	5,596 / 1,199	0.470	AST	0.590

7.7 鲁棒性的进一步解读

表 18 的完整结果可以按「表示」而非「模型」来读：

- 深度模型的输入是片段内最大值归一的 Log-Mel，因此对纯增益严格不变；传统模型的 MFCC 是绝对 dB，还包含 RMS，增益变化会直接移动特征分布，掉 0.7–3.4 个百分点。
- 噪声抬高了频谱的动态范围地板。以 `blues.00008` 为例，干净片段的 Log-Mel 最小值是 −80 dB，10 dB SNR 时变为 −46.3 dB；在 `ref=max` 归一化后，安静帧整体上移几十 dB，相当于强烈的分布外输入。这解释了为什么连「几乎听不见」的 30 dB 噪声也能让 CNN 大幅掉点。
- 片段长度同时改变上下文与时间尺度：3 s 片段被线性拉伸到 216 帧（1.66 倍），10 s 被压缩到 216 帧（2 倍）。CRNN 的 GRU 对时间尺度最敏感，3 s 时 Macro-F1 只剩 0.169；而 SVM/MLP 的统计量在更长窗口上反而更稳（10 s 提升约 5 个百分点）。

7.8 威胁与局限

构念效度　流派标签是人为构造的，同一首歌可能同时属于多个流派；模型学到的是「数据集标注者的偏好」，而不是音乐的本质。GTZAN 的标签噪声、FMA 的宽类别都放大了这一问题。

内部效度　尽管我们固定了划分、种子与预处理，GPU的非确定性仍可能带来 1–2 个百分点的波动（例如自适应平均池化的 CUDA 反向传播没有确定性实现[1]）。因此我们只对大于 3 个百分点的差异做解释；1–2 个百分点的差异视为噪声。

外部效度　两个数据集都以西方流行音乐为主，未覆盖中文流行、民族音乐、古典乐以外的非西方传统音乐；片段统一取中心 5 秒，未利用整首 30 秒的结构信息；扰动实验只覆盖增益、噪声、片段长度，未覆盖变速、混响、压缩失真与现场录音。

统计效度　GTZAN 测试集仅 147 首，比例指标的 95% 置信半径约 ±7 个百分点；FMA 为 1,199 首，约 ±2.8 个百分点。3 个种子只能给出粗略的方差估计，报告中的标准差应理解为「同一协议下的运行波动」，而不是总体置信区间。

实现层面　AST 的 16 kHz 与 FMA 原始 44.1/48 kHz 存在域差异，5 秒补零到 10.24 秒意味着一半输入是静音；受算力限制，AST 只做线性探针、未微调，其结果代表「冻结表征的线性可分性」而非 AST 的上限。

生成式任务的特有局限　生成式任务的可信度弱于判别式部分，原因有四。其一，音频级评估只在 **8** 首测试曲上统计，配对检验的统计功效很低，本文因此把所有 0.3 dB 量级的差异一律判为不显著，宁可保守也不宣称收益。其二，除 Token 预测模型跑了 3 个种子外，MusicGen 的前缀续写与 LoRA 适配都是单一随机种子，生成采样的方差未被估计。其三，受解码器直连路径的位置索引限制，续写长度统一截到 3 秒，无法评估长于 3 秒的结构连贯性；这一点已如实写进结果，未用” 跑通 3 秒” 冒充” 完成 5 秒续写”。其四，音频级距离用的是与这一首真实后续的 Log-Mel RMSE，它奖励” 贴近原曲” 而不是” 音乐性好”，对自由创作式的续写天然不利。这四条叠在一起意味着：本文能够可靠地说” 修正目标对齐带来了可测的改善”，但不足以支撑任何关于音乐质量的更强主张。

7.9 伦理与数据授权

GTZAN 与 FMA 的音频版权归各权利人所有，本文只使用其进行研究与课程演示，不再分发音频；仓库不包含原始音频与模型权重，只提交划分清单、代码与结果。模型可能强化「流派」这一西方音乐产业分类体系，在跨文化应用中需要重新审视标签体系与标注者构成。

7.10 可复现性总结

全部实验由命令行脚本驱动，随机源显式定种；划分、特征、预测与图表均可从仓库重新生成。自动化测试覆盖四条协议不变量；独立校验员对结果文件逐条复算，未发现泄漏或伪造。唯一不完全可复现的是 GPU 非确定性带来的细微数值差异，已在第 4 节说明。

7.11 对课程项目的启示

1. 协议比模型数量更重要：同一划分、同一批片段、训练集拟合预处理，才能让「谁更强」这句话有意义；
2. 先排查泄漏再谈提升：FMA的曲目级划分让 81% 的测试曲目与训练集共享艺术家，若不检查，任何「提升」都可能只是记住了乐队；
3. 鲁棒性与部署成本是模型选择的一部分：干净输入上的冠军在噪声下可能垫底，346 MB 的预训练模型也不适合实时场景；

[1] 具体为 `adaptive_avg_pool2d_backward_cuda`。

4. 可视化与失败案例比单一准确率更有解释力：混淆矩阵、置信度、时间轴让观众直观看到模型「什么时候会翻车」。

8 结论与展望

本文在一个统一、可复现的协议下，系统比较了从 SVM 到预训练 AST 的六类音乐流派分类模型，并在 GTZAN（985 首）与 FMA-small（7,994 首）上验证结论的稳定性。我们的核心发现可以概括为四点：预训练表示带来最大的单点收益；深度模型在干净输入上领先，但在噪声与短片段下并不比传统方法稳健；小数据上的轻量 Transformer 容易过拟合；数据划分中的艺术家泄漏足以让结论虚高近 10 个百分点。

同一套思路也适用于生成式任务：用同一套 EnCodec Token 与同一份按艺术家分离的划分，先验证" 预测真实未来" 能否被度量，再验证" 前缀续写" 能否在 8 GB 卡上跑通。结论是分层的：Token 级预测可复现、可比较，但受自回归误差累积限制；预训练生成模型的推理可以量化成本，而参数高效适配的收益必须与显存预算一起报告，不能把" 跑通推理" 写成" 完成微调"。

作为课程项目，本文的另一个目标是把「做研究」的流程完整走一遍：先审计环境与数据，再固定协议、做消融、分析失败案例，最后把结果写成可追溯的报告并开源。我们刻意保留了几个负结果（优化消融无效、Transformer 失利、噪声下全面崩塌），因为诚实报告失败模式比堆叠准确率更有教学价值。

未来工作可以从四个方向展开：

1. 更长的上下文：当前只使用中心 5 秒片段，未来可利用整首 30 秒或滑动窗口聚合，并系统研究片段长度与性能的关系；
2. 更强的鲁棒性训练：在训练中加入噪声、增益、变速与混响增强，或采用固定 dB 参考、每帧 CMVN 等对增益/噪声更稳健的归一化；
3. 知识蒸馏与轻量化：把 AST 的表征蒸馏到 CNN/CRNN，在保留大部分收益的同时把推理成本降回毫秒级；
4. 跨文化与多模态：引入非西方音乐、歌词与元数据，重新审视「流派」标签体系，并扩展到乐器识别、情绪识别等相关任务；
5. 更长的生成 **horizon**：当前 Token 预测只有 1 秒，续写只有 5 秒；下一步可把预测器与生成模型级联（预测器提词、生成模型续写），并用规模化的主观评分检验" 好听"。

参考文献

[1] G. Tzanetakis and P. Cook. Musical genre classification of audio signals. *IEEE Transactions on Speech and Audio Processing*, 10(5):293–302, 2002.

[2] B. L. Sturm. The GTZAN dataset: Its contents, its faults, their effects on evaluation, and its future use. *arXiv:1306.1461*, 2013.

[3] M. Defferrard, K. Benzi, P. Vandergheynst, and X. Bresson. FMA: A dataset for music analysis. In *ISMIR*, 2017.

[4] K. Choi, G. Fazekas, M. Sandler, and K. Cho. Convolutional recurrent neural networks for music classification. In *ICASSP*, 2017.

[5] Y. Gong, Y.-A. Chung, and J. Glass. AST: Audio Spectrogram Transformer. In *Interspeech*, 2021.

[6] A. Vaswani et al. Attention is all you need. In *NeurIPS*, 2017.

[7] J. F. Gemmeke et al. Audio Set: An ontology and human-labeled dataset for audio events. In *ICASSP*, 2017.

[8] D. S. Park et al. SpecAugment: A simple data augmentation method for automatic speech recognition. In *Interspeech*, 2019.

[9] B. McFee et al. librosa: Audio and music signal analysis in Python. In *SciPy*, 2015.

[10] F. Pedregosa et al. Scikit-learn: Machine learning in Python. *JMLR*, 12:2825–2830, 2011.

[11] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.

[12] C. Szegedy et al. Rethinking the inception architecture for computer vision. In *CVPR*, 2016.

[13] M. A. Casey et al. Content-based music information retrieval: Current directions and future challenges. *Proceedings of the IEEE*, 96(4):668–696, 2008.

[14] S. Dieleman and B. Schrauwen. End-to-end learning for music audio. In *ICASSP*, 2014.

[15] Q. Kong et al. PANNs: Large-scale pretrained audio neural networks for audio pattern recognition. *IEEE/ACM TASLP*, 28:2880–2894, 2020.

[16] J. S. Downie. Music information retrieval. *Annual Review of Information Science and Technology*, 37:295–340, 2003.

[17] D. Bogdanov et al. Essentia: An audio analysis library for music information retrieval. In *ISMIR*, 2013.

[18] T. Bertin-Mahieux et al. The Million Song Dataset. In *ISMIR*, 2011.

[19] E. J. Humphrey, J. P. Bello, and Y. LeCun. Feature learning and deep architectures: New directions for music informatics. *Journal of Intelligent Information Systems*, 41:461–481, 2013.

[20] S. Hershey et al. CNN architectures for large-scale audio classification. In *ICASSP*, 2017.

[21] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi. High fidelity neural audio compression. *Transactions on Machine Learning Research*, 2023.

[22] Z. Borsos et al. AudioLM: a language modeling approach to audio generation. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2023.

[23] J. Copet, F. Kreuk, I. Gat, T. Remez, D. Kant, G. Synnaeve, Y. Adi, and A. Défossez. Simple and controllable music generation. In *NeurIPS*, 2023.

[24] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022.

[25] K. Kilgour, M. Zuluaga, D. Roblek, and M. Sharifi. Fréchet Audio Distance: a reference-free metric for evaluating music enhancement algorithms. In *Interspeech*, 2019.

A 附录：数据、超参与复现细节

A.1 数据集统计

表 **25:** 两个数据集的实际使用统计。

数据集	清洗后曲目	流派数	划分（train/val/test）
GTZAN	985	10	689 / 149 / 147
FMA-small	7994	8	5596 / 1199 / 1199

GTZAN 原始 1,000 首，剔除 1 首损坏（`jazz.00054`）与 14 首重复副本后为 985 首；FMA-small 原始 8,000 首，剔除 6 首损坏后为 7,994 首。FMA 的采样率为 44.1 kHz（7,569 首）、48 kHz（411 首）与 22.05 kHz（14 首）混合，统一重采样到 22.05 kHz；GTZAN 全部为 22.05 kHz。

A.2 完整超参数

表 **26:** 模型与训练超参数（两套数据集的差异仅在类别数）。

模型	超参数
SVM	RBF 核，$C = 10$，γ =scale，probability=True（Platt 校准）
MLP	隐藏层 256–128，ReLU，$\alpha = 10^{-4}$，batch 64，lr 10^{-3}，max_iter 300，early stopping（15% 内部验证，patience 20）
CNN	4 个卷积块（32/64/128/128），每块 2 层 3×3 卷积 + BN + ReLU + 2×2 池化；AdaptiveAvgPool(2,4)，dropout 0.3，FC(1024,10)
CRNN	3 个卷积块（32/64/128），池化 $(2,2)/(2,2)/(2,1)$；BiGRU(96)；时间平均 + FC(192,10)
Transformer	$d = 128$，4 头，2 层 Pre-LN，FFN 256，dropout 0.1，可学习位置编码
共同训练设置	AdamW，lr 10^{-3}，weight decay 10^{-4}，余弦退火，batch 32，最多 30 epoch，按 val Macro-F1 早停（patience 8）
AST	冻结主干，StandardScaler + LogisticRegression，$C \in \{0.001, \ldots, 10\}$ 由 val 选择，16 kHz、中心 5 秒、补零 10.24 秒

A.3 鲁棒性完整变化表

表 **27:** 相对 clean 的准确率变化（百分点，GTZAN seed 42）。

模型	3s	10s	0.5×	1.5×	10dB	0dB
SVM	-12.9	+4.1	-0.7	-3.4	-37.4	-40.8
MLP	-10.9	+6.1	-2.7	-0.7	-36.7	-42.2
CNN	-34.0	-21.1	+0.0	+0.0	-53.7	-62.6
CRNN	-53.1	-21.8	+0.0	+0.0	-49.0	-57.8
Transformer	-3.4	+3.4	+0.0	+0.0	-33.3	-41.5

A.4 AST 的 C 网格

表 **28:** AST 线性头的验证集 C 网格（FMA-small）。

C	val Macro-F1
0.001	0.5782
0.003	0.5790
0.01	0.5564
0.03	0.5439
0.1	0.5079
0.3	0.4890
1.0	0.4691
3.0	0.4528
10.0	0.4511

A.5 仓库结构与复现命令

```
MUSIC_AI/
├── configs/            dataset.yaml, baseline.yaml, experiments.yaml
```

```
├── data/               raw/, features/{gtzan,fma}/, splits/{gtzan,fma}/, fma/
├── src/
│   ├── data/           prepare_dataset.py, prepare_fma.py, audio_features.py
│   ├── models/         cnn.py (GenreCNN / CRNN / SmallTransformer)
│   ├── forecasting/    EnCodec Token 化、预测模型、解码与出图
│   ├── generation/     MusicGen 续写推理与 LoRA 适配
│   ├── train_classical.py, train_cnn.py, train_ast.py
│   ├── evaluate.py, predict.py, error_analysis.py, robustness.py, timeline.py
│   └── metrics.py
├── scripts/            extract_features.py, run_fma.sh, fma_ablation.sh,
│                       ast_fma_fast.py, robustness_flips.py, …
├── app/demo.py         Streamlit 演示
├── tests/test_protocol.py
├── checkpoints/        gtzan/, fma/, fma_leak/, fma_ablation/,
│                       token_prediction/, generation/
├── results/            gtzan/, fma/, fma_leak/, fma_ablation/,
│                       token_prediction/, generation/（与权重同名）
├── docs/                MD 文档 + report/（LaTeX 源与 PDF）
└── README.md
```

```
PY=/home/weizai0v0/.apps/anaconda3/envs/DataAnalysis/bin/python
# GTZAN MVP
$PY -m src.data.prepare_dataset --download --extract --audit --split
$PY scripts/extract_features.py --workers 8
$PY -m src.train_classical --models svm mlp --seeds 42 123 2024
$PY -m src.train_cnn --model cnn --seeds 42 123 2024 --epochs 30
$PY -m src.train_cnn --model crnn --seeds 42 123 2024 --epochs 30
$PY -m src.train_cnn --model transformer --seeds 42 123 2024 --epochs 30
$PY -m src.evaluate
# FMA-small 主实验
bash scripts/run_fma.sh 42 123 2024
$PY scripts/ast_fma_fast.py --reuse-feats --half
# 泄漏对照
MUSIC_FEATURES=data/features/fma/features_tracklevel.npz \
  MUSIC_RESULTS=results/fma_leak MUSIC_CKPT=checkpoints/fma_leak \
  $PY -m src.train_cnn --model cnn --seeds 42 --epochs 30
# 生成式任务：Token 预测
bash scripts/run_token.sh 42 123 2024
$PY -m src.forecasting.decode_tokens --model transformer --seed 42
$PY -m src.forecasting.plot_tokens
# 生成式任务：MusicGen 前缀续写（约 5.8 GB 权重，一次性下载）
$PY -m src.generation.run_musicgen --tag base --n 6 --seconds 5
# 生成式任务：LoRA 适配（可选，8 GB 显存先验证再跑）
$PY -m src.forecasting.prepare_tokens --prefix-s 5 --target-s 5 --subdir _ft \
    --windows-per-track 2 --shard-size 512 --encode-batch 16
$PY -m src.generation.finetune_lora --steps 300
# 报告
```

```
cd docs/report && bash build.sh
# 测试
$PY -m pytest -q tests
```

A.6 结果文件说明

表 **29:** 主要结果文件。

文件	内容
`results/gtzan/metrics.csv`	GTZAN 逐 seed 指标（16 行，含 AST）
`results/gtzan/_eval.json`	混淆矩阵、逐类 F1、训练曲线
`results/gtzan/predictions/*.csv`	逐样本预测与 10 类概率
`results/gtzan/robustness.csv`	5 模型 × 7 扰动的 35 行结果
`results/fma/metrics.csv`	FMA 逐 seed 指标（16 行，含 AST）
`results/fma/ast_metrics.json`	AST 迁移结果与成本
`results/fma_leak/metrics.csv`	曲目级划分的泄漏对照
`results/token_prediction/metrics.csv`	Token 预测逐 seed 指标
`results/token_prediction/decode.json`	解码回音频的对照样例
`results/generation/metrics_base.json`	MusicGen 原始模型续写与成本
`docs/audit_report.md`	独立只读审计报告